Self-Supervised Locality Preserving Low-Pass Graph Convolutional Embedding for Large-Scale Hyperspectral Image Clustering

Yao Ding, Zhili Zhang, Xiaofeng Zhao, Yaoming Cai, *Member, IEEE*, Siye Li, Biao Deng, and Weiwei Cai, *Member, IEEE*

***Abstract*— Due to prior knowledge deficiency, large spectral variability, and high dimension of hyperspectral image (HSI), HSI clustering is extremally a fundamental but challenging task. Deep clustering methods have achieved remarkable success and have attracted increasing attention in unsupervised HSI classification (HSIC). However, the poor robustness, adaptability, and feature presentation limit their practical applications to complex large-scale HSI datasets. Thus, this article introduces a novel self-supervised locality preserving low-pass graph convolutional embedding method (L2GCC) for large-scale hyperspectral image clustering. Specifically, a spectral–spatial transformation HSI preprocessing mechanism is introduced to learn superpixel-level spectral–spatial features from HSI and reduce the number of graph nodes for subsequent network processing. In addition, locality preserving low-pass graph convolutional embedding autoencoder is proposed, in which the low-pass graph convolution and layerwise graph attention are designed to extract the smoother features and preserve layerwise locality features, respectively. Finally, we develop a self-training strategy, in which a self-training clustering objective employs soft labels to supervise the clustering process and obtain appropriate hidden representations for node clustering. L2GCC is an end-to-end training network, which is jointly optimized by graph reconstruction loss and self-training clustering loss. On Indian Pines, Salinas, and University of Houston 2013 datasets, the clustering accuracy overall accuracies (OAs) of the proposed L2GCC are 73.51%, 83.15%, and 64.12%, respectively.**

***Index Terms*— Deep clustering, hyperspectral image clustering, layerwise graph attention, low-pass graph convolution, self-training.**

NOMENCLATURE

n	Number of nodes contained in the graph.
d_i	D of $\boldsymbol{X}$ and the ith latent feature.
c	Number of clusters.
W	Learnable weight matrix.
$\boldsymbol{S}_i$	ith superpixel.
$\boldsymbol{X}$	Node feature matrix.
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	Graph $\mathcal{G}$ with edges set $\mathcal{E}$ and nodes set $\mathcal{V}$.
$\boldsymbol{A}$, $\hat{\boldsymbol{A}}_{ij}$	Adjacency matrix of $\mathcal{G}$. The adjacency matrix of a reconstructed graph.
$\boldsymbol{D} = \text{diag}(d_1, \ldots, d_n)$	Degree matrix of A.
$\boldsymbol{L}$, $\boldsymbol{L}_{\text{sym}}$	Graph Laplacian symmetric and normalized Laplacian of $\mathcal{G}$.
$\boldsymbol{U}$	Eigenvector matrix of $\boldsymbol{L}_{\text{sym}}$.
$\cdot \parallel \cdot$	Concatenation operation.
$\boldsymbol{Z}_i$	ith layer encoder output.
$\lambda_{\max}$, λ_i	Largest eigenvalue and the ith eigenvalue.
$\mathcal{N}(\cdot)$	Neighborhood of $\bullet$.
$\odot$	Elementwise product.
P, Q	Target distribution. The soft labels distribution.
η	Learning rate.

I. INTRODUCTION

HYPERSPECTRAL image (HSI) acquired by remote sensing system is composed of hundreds of contiguous and narrow spectral bands with abundant spatial–spectral information in the electromagnetic spectrum [1], [2], [3]. Due to its unique advantages, HSI has attracted lots of attention and has been widely applied in various fields, including military reconnaissance, urban mapping, biochemical detection, forest fire detection, and target recognition [4], [5], [6].

As one of the fundamental and common techniques for HSI practical application, HSI classification (HSIC), aiming to divide each pixel contained in the image into different land covers, has been widely studied. Recently, many supervised or semisupervised deep learning HSIC methods have achieved promising performances [7], [8]. However, these methods rely on annotated samples to train the network. In practice, it is a high labor-intensive and time-consuming progress to obtain the labeled HSI pixels, which is a problem that must be considered in the application of the HSIC method. To solve the aforementioned problem, the unsupervised clustering method, aiming to assign similar

Manuscript received 30 April 2022; revised 3 July 2022; accepted 1 August 2022. Date of publication 16 August 2022; date of current version 31 August 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 41404022. *(Corresponding author: Zhili Zhang.)*

Yao Ding, Zhili Zhang, Xiaofeng Zhao, Siye Li, and Biao Deng are with the School of Optical Engineering, Xi'an Research Institute of High Technology, Xi'an 710025, China (e-mail: dingyao.88@outlook.com; 157918018@qq.com; xifezhao@163.com; lisiye97@163.com; djm202@163.com.)

Yaoming Cai is with the School of Computer Science, China University of Geosciences, Wuhan 430074, China (e-mail: caiyaom@cug.edu.cn).

Weiwei Cai is with the School of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi 214122, China (e-mail: vivitsai@ieee.org).

Digital Object Identifier 10.1109/TGRS.2022.3198842

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

pixels into unique land-cover classes with no prior labels, has attracted increasing attention. Compared with supervised or semisupervised methods, HSI clustering is often a fundamental but challenging task, due to prior knowledge deficiency, large spectral variability, and high dimension of HSI [9], [10].

In the past several decades, various clustering methods have been designed for HSI clustering [11], [12], [13], [14]. In the early stage, some traditional clustering algorithms developed for natural images have been applied to HSI clustering, such as k-means [15], spectral clustering (SC) [16], fuzzy c-means (FCM) [17], and independent component analysis (ICA) model [18]. However, these algorithms only extract spectral information, and the spatial information contained in HSI is ignored, which limits their performances. Thus, to fully utilize the spectral–spatial information of HSI and improve the clustering accuracy, some spectral–spatial HSI clustering methods [19], [20], [21] are designed, and better performances have been achieved than that of only spectral-based methods. For example, Elhamifar and Vidal [22] proposed FCM-S1, in which the spatial-contextual information is incorporated. Furthermore, to solve the high-dimensional problem of HSI, some sparse subspace clustering (SSC) algorithms [23], [24], [25] have appeared, which can cluster the pixels in the low-dimensional subspace by learning the coefficient representation matrix and constructing the affinity matrix. In addition, Zhai *et al.* [26] integrated the total variation and collaborative subspace clustering (CSC) into a unified framework to present the spectral–spatial information of HSIs. A scalable graph-based clustering method named SGCNR was designed by Chen and Zhang [27] using nonnegative relaxation to present HSI features.

Unfortunately, the aforementioned methods are shallow classifiers based on the handcrafted feature or raw feature spaces, which cannot learn the high-level features of HSI and are not suitable for characterizing the large spectral variability and the complex internal structure of HSI datasets effectively [28], [29], [30]. Thus, some deep learning techniques have been applied to overcome the limitation. With the successful application of deep learning-based clustering in natural image clustering, some deep learning clustering methods have been proposed to tackle the HSI clustering task. We can divide most of the available deep HSI clustering methods into two steps, including high-level feature extraction adopting deep learning techniques and traditional clustering. In [31], a graph regularized residual subspace clustering network (GR-RSCNet) was introduced, where the robust nonlinear affinity and deep spectral–spatial representation were jointly learned to present the complex features of HSI in a deep neural network. Cai *et al.* [32] presented a neighborhood contrastive subspace clustering (NCSC) network, a scalable and robust DSC approach, for unsupervised classification of large hyperspectral images. Inspired by the autoencoder technique in the computer vision field, some researchers have introduced autoencoder into the clustering method for spatial–spectral feature learning. For instance, a spatial–spectral subspace clustering network (DS^{3}C-Net) was designed by Lei *et al.* [11] to extract spatial–spectral features by collaborative constraint and the multiscale autoencoder. Hou *et al.* [33] introduced contrast learning [34] into HSI features learning, and the unlabeled samples were utilized to alleviate the label information insufficient problem. Nevertheless, the aforementioned methods are not designed specifically for large-scale HSI clustering, and the clustering performances on complex large-scale HSI dataset clustering should be improved. In addition, due to labeled data deficiency, large spectral variability, and high dimension of HSIs, most of the existing deep clustering methods cannot effectively learn remarkable and adaptive high-level features for HSI clustering, which leads to poor clustering accuracy. Finally, most of the deep clustering methods ignore the layerwise local features, which results in poor adaptability to different HSI datasets.

To address these drawbacks, we propose a novel self-supervised locality preserving low-pass graph convolutional embedding method (L2GCC) for large-scale hyperspectral image clustering. Specifically, a spectral–spatial transformation HSI preprocessing mechanism is introduced to learn superpixel-level spectral–spatial features from HSI and reduce the number of graph nodes for subsequent network processing. In addition, we design a low-pass graph convolutional embedded autoencoder to learn a hidden representation of the graph by training the inner product decoder to reconstruct the graph, with which a smoother graph compact representation can be learned. Furthermore, we proposed a layerwise graph attention module to preserve the local features of each layer and improve the clustering adaptability to different HSI datasets. Finally, we develop a self-training strategy, in which a self-training clustering objective employs soft labels to supervise the clustering process and obtain appropriate hidden representations for node clustering. In L2GCC, we jointly integrate locality preserving low-pass graph convolution autoencoder and self-training clustering into a single end-to-end network to achieve mutual benefit by adopting jointly optimizing loss during network training. In summary, the main contributions of L2GCC are summarized as follows.

1) We design a locality preserving low-pass graph convolutional embedding autoencoder, in which the low-pass graph convolution and layerwise graph attention are designed to extract the smoother features and locality features, respectively.
2) We introduce a self-training clustering mechanism to refine the clustering results and solve the labeled samples deficiency problem of HSIs, and soft labels generated from graph embedding are proposed to supervise the self-training clustering process.
3) The locality preserving low-pass graph convolutional autoencoder presents self-training clustering that is jointly integrated into a single network to achieve mutual benefit by adopting jointly optimizing loss. Compared with state-of-the-art clustering baselines, the experimental results verify our L2GCC superiority.

II. Background

A. Notations and Descriptions

In this article, all vectors are represented in lowercase and bold and all matrices are denoted in uppercase. n, d, and c are adopted to represent the size of nodes and dimension of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

HSI (node-set number of clusters). The input node set $\{x_i\}_{i=1}^{n}$ can be expressed as

$$\boldsymbol{X} = [x_1, x_2, \ldots, x_n] \in \mathbb{R}^{n\times d}. \tag{1}$$

Let $\mathcal{V}$ be the nodes set, ε be the edge set between nodes, and $\boldsymbol{X} \in \mathbb{R}_{n\times d}$ be the node feature matrix, and then, the graph can be denoted as $\mathcal{G} = (\mathcal{V}, \varepsilon)$, in which the adjacency matrix is denoted as $\boldsymbol{A} \in R_{n\times n}$ and the corresponding degree matrix is presented as $\boldsymbol{D} \in \mathbb{R}_{n\times n}$. Furthermore, $\cdot\|\cdot$ denotes the concatenation operation. The main notations adopted in this article and their descriptions are demonstrated in the Nomenclature.

B. Convolution on Graph

We can explain the graph convolution operation as both spatial operators [35] and spectral operators [36]. In this article, we define the convolution operator from the frequency, which presents the graph signals in the spectral domain. Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \boldsymbol{A})$, where $\mathcal{V} = \{v_i\}_{i=1,\ldots,n}$ denotes the set of the vertex with n nodes and m channels, $\varepsilon = \{e_{ij}\}_{i,j=1,\ldots,n}$ is the set of edges between node (superpixel), the adjacency matrix of the graph is $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, and the weight of the edge between vertices i and j is A_{ij}, in which $A_{ij} = 1$, if $\{v_i, v_j\} \in \varepsilon$, indicating that there is an edge between nodes v_i and v_i; otherwise, $A_{ij} = 0$. The node feature matrix is presented as $\boldsymbol{X} = \{\boldsymbol{x}_1; \cdots; \boldsymbol{x}_n\}$, where $\boldsymbol{x}_i \in \mathbb{R}^m$ is the feature value associated with node v_i, n denote the number of nodes contained in the graph, and m is the dimension of ith latent feature. In the graph, we can denote the topological structure of the graph $\mathcal{G}$ by constructing an adjacency matrix $\boldsymbol{A}$. The mathematical representation of the normalized graph can be defined as $\boldsymbol{L} = \boldsymbol{I_n} - \boldsymbol{D}^{-(1/2)}\boldsymbol{A}\boldsymbol{D}^{-(1/2)}$, where $\boldsymbol{D}$ is the degree matrix, $\boldsymbol{D_{ii}} = \sum_j(\boldsymbol{A_{i,j}})$. $\boldsymbol{L}$ is real symmetric positive semidefinite, and we can factor $\boldsymbol{L}$ as $\boldsymbol{L} = \boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U^T}$, where $\boldsymbol{U} = [\boldsymbol{u_0}, \boldsymbol{u_1}, \ldots, \boldsymbol{u_{n-1}}] \in \mathbb{R}^{N\times N}$ is the eigenvectors matrix of $\boldsymbol{L}$, i.e., $\boldsymbol{U}\boldsymbol{U^T} = \boldsymbol{E}$, where $\boldsymbol{E}$ is the identity matrix, $\boldsymbol{\Lambda}$ is the diagonal matrix of eigenvalues, and $\boldsymbol{\Lambda_{ii}} = \boldsymbol{\lambda_i}$, $\boldsymbol{\lambda_i}$ denotes the eigenvalue. Given a graph signal $\boldsymbol{X} \in \mathbb{R}^{N\times M}$ (graph node feature matrix), the Fourier transform of $\boldsymbol{X}$ can be expressed as $\mathcal{F}(\boldsymbol{X}) = \boldsymbol{U^T}\boldsymbol{X}$, and the inverse graph Fourier transform can be expressed as $\boldsymbol{\mathcal{F}^{-1}}(\hat{\boldsymbol{X}}) = \boldsymbol{U}\hat{\boldsymbol{X}}$, where $\hat{\boldsymbol{X}}$ is the result of Fourier transform. The graph Fourier transform projects the graph feature vector into the orthogonal space, in which the basis is composed of the eigenvector of the normalized graph Laplace operator. The elements in $\hat{\boldsymbol{X}}$ are the coordinates projected by $\boldsymbol{X}$ in the orthogonal space, and $\boldsymbol{X}$ can be expressed as $\boldsymbol{X} = \sum_{\boldsymbol{i}} \hat{\boldsymbol{x}}_{\boldsymbol{i}}\boldsymbol{u}_{\boldsymbol{i}}$. We can express the graph convolution of $\boldsymbol{X}$ with a graph filter $\boldsymbol{g}$ as

$$\begin{aligned}\boldsymbol{g} * \boldsymbol{X} &= \mathcal{F}^{-1}(\mathcal{F}(\mathrm{x}) \odot \mathcal{F}(\mathrm{g})) \\ &= \boldsymbol{U}\left(\boldsymbol{U}^T\boldsymbol{X} \odot \boldsymbol{U}^T\boldsymbol{g}\right).\end{aligned} \tag{2}$$

If the graph filter is expressed as $\boldsymbol{g}_\theta = \mathrm{diag}(\boldsymbol{U}^T\boldsymbol{g})$, then (2) can be simplified as

$$\boldsymbol{g}_\theta * \boldsymbol{X} = \boldsymbol{U}\boldsymbol{g}_\theta(\boldsymbol{\Lambda})\boldsymbol{U}^{\mathrm{T}}\boldsymbol{X}. \tag{3}$$

The spectral-based graph convolution algorithms all follow the definition of (3). The graph filter is important to spectral-based graph convolution algorithms, and graph filter designing can determine the spectral-based graph convolution algorithms' performances. The aggregation operation in graph convolutional network (GCN) [37] is expressed as

$$\boldsymbol{X}_{i+1} = \sigma\left(\boldsymbol{D}^{-\frac{1}{2}}\hat{\boldsymbol{A}}\boldsymbol{D}^{-\frac{1}{2}}\boldsymbol{X}_i\boldsymbol{W}_i\right) \tag{4}$$

where $\boldsymbol{X}_i$ is the output feature matrix of the ith layer and $\boldsymbol{X}_0 = \boldsymbol{X}$. Equation (4) shows the node feature transformation of GCN from $\boldsymbol{X}_i \in \mathbb{R}^{n\times c_i}$ to $\boldsymbol{X}_{i+1} \in \mathbb{R}^{n\times c_{i+1}}$. σ is a nonlinear activation function and $\boldsymbol{W}_i \in \mathbb{R}^{c_i\times c_{i+1}}$ is a learnable weight matrix to realize the linear transformation on the feature matrix. $\hat{\boldsymbol{A}} = \boldsymbol{A} + \boldsymbol{I}$, and $\boldsymbol{D} = \mathrm{diag}(d_1, \ldots, d_n,)$ is the degree matrix to performing normalization on $\hat{\boldsymbol{A}}$.

C. Attention Convolution on Graph

Given a graph G, the feature eigenvector of each node is $\boldsymbol{x}_i \in \mathbb{R}^m$. A new eigenvector $\boldsymbol{x_i}' \in \mathbb{R}^{m'}$ can be obtained by linear transformation for $\boldsymbol{x}_i$, and we can express the operation process as

$$\boldsymbol{x_i}' = \boldsymbol{W}\boldsymbol{x}_i, \quad \boldsymbol{W} \in \mathbb{R}^{m\times m'}. \tag{5}$$

If the node j is a neighbor of node i, we can adopt the attention mechanism to calculate the importance of node j to node i, the attention score e_{ij} can be calculated as

$$e_{ij} = \left(\sigma\left(\boldsymbol{a}^T\left[\boldsymbol{W}\boldsymbol{x}_i||\boldsymbol{W}\boldsymbol{x}_j\right]\right)\right) \tag{6}$$

where $||$ denotes the concatenation operation, $\boldsymbol{a}^T \in \mathbb{R}^{2m'}$ is the parameter vector, and σ is a nonlinear layer, i.e., LeakyReLU.

e_{ij} can be converted to the following form through a softmax function

$$\alpha_{ij} = \mathrm{softmax}(e_{ij}) = \frac{\exp\left(\sigma\left(\boldsymbol{a}^T\left[\boldsymbol{W}\boldsymbol{x}_i||\boldsymbol{W}\boldsymbol{x}_j\right]\right)\right)}{\sum_{k\in N_i}\exp\left(\sigma\left(\boldsymbol{a}^T\left[\boldsymbol{W}\boldsymbol{x}_i||\boldsymbol{W}\boldsymbol{x}_j\right]\right)\right)}. \tag{7}$$

The mathematical expression of graph convolution output of graph attention network (GAT) [38] is

$$\boldsymbol{x_i}' = \sigma\left(\sum_{j\in n_i}\alpha_{ij}\cdot\boldsymbol{W}^T\boldsymbol{x}_j\right) \tag{8}$$

where σ is the activation function, a_{ij} is the learned attention weight, and n_i denotes the number of neighbor nodes of node i.

D. Graph Autoencoder

The graph autoencoder (GAE) [39] utilizes multilayer GCN to learn the hidden presentation of graph nodes embeddings and the inner products between two nodes are computed; then, they are mapped into a probability space to calculate the similarities through a sigmoid function and the graph is reconstructed by computing the graph reconstructed loss. The network architectures of GAEs [39], [40], [41], [42] are often asymmetric, while we can employ graph sharpening to designs the symmetric architecture. For instance, adversarial learning is incorporated into GAE to enhance the robustness in

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

adversarial regularized graph autoencoder [42]. In GAE with adaptive graph convolution [41], the high-order convolution operator is introduced into GAE to improve the presentation capacity.

III. Proposed Method

A. Problem Definition and Overall Framework

Given a nondirected graph $\mathcal{G}$, the goal is to classify the nodes into c clusters, i.e., $\{C_1, C_2, \ldots, C_c\}$, by using a self-supervised locality preserving low-pass graph convolutional embedding autoencoder to learn the latent representation $\boldsymbol{Z}$. Therefore, the nodes in a single cluster contain similar spectral feature values and spatial structure.

From the flow diagram of L2GCC in Fig. 1, we can partition our method into three parts: including pixel-to-region transformation (PRT), locality preserving low-pass graph convolutional embedding encoder, and graph convolution embedding decoder.

1) *PRT:* The HSI is segmented into local superpixel, and the mean spectral value of pixels contained in the superpixel is treated as the feature value, with which the number of graph node is reduced.
2) *Locality Preserving Low-Pass Graph Convolutional Embedding Encoder:* The node feature and the graph structure are taken as the input of the network. The hidden representation $\boldsymbol{Z}$ is learned by the locality preserving low-pass graph convolutional embedding autoencoder, in which the low-pass GCN and layerwise GAT are designed to extract the smoother features and locality features, respectively.
3) *Graph Convolutional Embedding Decoder:* The proposed method contains two decoders. Namely, the graph is reconstructed by minimizing the Kullback–Leibler (KL) divergence between the adjacency matrix $\boldsymbol{A}$ distribution and the adjacency matrix $\hat{\boldsymbol{A}}$ of reconstructed graph distribution, and self-training clustering can be performed by minimizing the KL divergence between the target distribution $\boldsymbol{P}$ and the soft label's distribution $\boldsymbol{Q}$.

The HSI preprocessing, graph convolutional embedding autoencoder, and node clustering are joined in a unified network, and each component benefits the others. Therefore, the proposed network is an end-to-end training network.

B. Locality Preserving Low-Pass Graph Convolutional Embedding Encoder

To partition the graph nodes into different clusters, the basic assumption is that the nodes in the same cluster are supposed to be closely related, that is, they have similar spatial–spectral features. Therefore, the latent representation $\boldsymbol{Z}$ should be smooth on the graph manifold.

Assumption 1: In the ideal case, the distance within clusters is the smallest and the distance between clusters is the largest, namely

$$\begin{cases} \min \sum_{i=1}^{m}\sum_{j=1}^{m} d(i,j) = \min \sum_{i=1}^{m}\sum_{j=1}^{m} \left\| f(x_i) - f\left(x_j\right)\right\|_2^2 \\ \max \sum_{\alpha=1}^{c}\sum_{\beta=1}^{c} d(\alpha,\beta) = \max \sum_{\alpha=1}^{c}\sum_{\beta=1}^{c} \left\| y(c_\alpha) - y\left(c_\beta\right)\right\|_2^2 \end{cases} \tag{9}$$

where $\boldsymbol{x}_i$ is the node feature of node i, $f(\cdot)$ is the mapping that attempts to extract node's optimal spatial–spectral representation, m is the number of nodes contained in a cluster, $y(\cdot)$ is the function to calculate the central feature of the cluster, and c is the number of clusters (classes).

1) Low-Pass Graph Convolutional Filter: A graph signal (node feature) is smooth if the presentation of neighbor nodes on the graph contains similar features. Given the graph signal $\boldsymbol{X}$, we can measure the graph signal $\boldsymbol{X}$ smoothness by calculating the Rayleigh quotient [43], i.e.,

$$R(\boldsymbol{L}, \boldsymbol{X}) = \frac{\boldsymbol{X}^\top \boldsymbol{L}\boldsymbol{X}}{\boldsymbol{X}^\top \boldsymbol{X}} = \frac{\sum_{(x_i,x_j)\in E}\left(x_i - x_j\right)^2}{\sum_{i\in V} x_i^2} \tag{10}$$

where $\boldsymbol{L}$ is the graph Laplacian and x_i is the ith node feature. As stated by (10), smooth signals are supposed to have similar values on nearby nodes, and the smoother signals contain a lower Rayleigh quotient.

Theorem 1: The smoother signals can be obtained by filtering high-frequency basis signals and preserving low-frequency basis signals.

Following the principle of Theorem 1, we can design a low-pass graph filter to process the signals of a graph for node clustering. We can express a low-pass graph filter in many forms. In this article, the frequency response function of low-pass graph filter is

$$p(\lambda_i) = 1 - k\lambda_i \tag{11}$$

where k is a real value and λ_i denotes the ith eigenvalues. Referring to the frequency response function $p(\cdot)$ in (11), the mathematical of graph filter $\boldsymbol{G}$ is expressed as

$$\boldsymbol{G} = \boldsymbol{U}\, p(\boldsymbol{\Lambda})\boldsymbol{U}^{-1} = \boldsymbol{U}(I - k\boldsymbol{\Lambda})\boldsymbol{U}^{-1} = I - k\boldsymbol{L} \tag{12}$$

Theorem 2: Given a graph filter $\boldsymbol{G}$, if its frequency response function $p(\lambda)$ is nonnegative and nonincreasing for all λ_i, then given any signal $\boldsymbol{X}$ and the filtered $\tilde{\boldsymbol{X}} = \boldsymbol{G}\boldsymbol{X}$, we always have

$$R\left(\tilde{\boldsymbol{X}}, \boldsymbol{X}\right) \le R(\boldsymbol{L}, \boldsymbol{X}). \tag{13}$$

According to Theorem 2, the filtered node features contain similar feature values in each dimension if such a low-pass graph filter is applied on the graph signal, which is easier for subsequent network clustering. Perform graph convolution on the node feature matrix $\boldsymbol{X}$, the filtered feature $\tilde{\boldsymbol{X}}$ can be obtained as

$$\begin{aligned} \tilde{\boldsymbol{X}} &= \boldsymbol{G}\boldsymbol{X} = \boldsymbol{U}\, p(\boldsymbol{\Lambda})\boldsymbol{U}^{-1} \cdot \boldsymbol{U}\boldsymbol{p} \\ &= \sum_{i=1}^{n} (1 - k\lambda_i) c_i \boldsymbol{u}_i = \sum_{i=1}^{n} c_i' \boldsymbol{u}_i \end{aligned} \tag{14}$$

where $\tilde{\boldsymbol{X}} = [\tilde{\boldsymbol{x}}_1, \tilde{\boldsymbol{x}}_2, \ldots, \tilde{\boldsymbol{x}}_n]^T$ and c_i is the coefficient of the ith eigenvector $\boldsymbol{u}_i$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

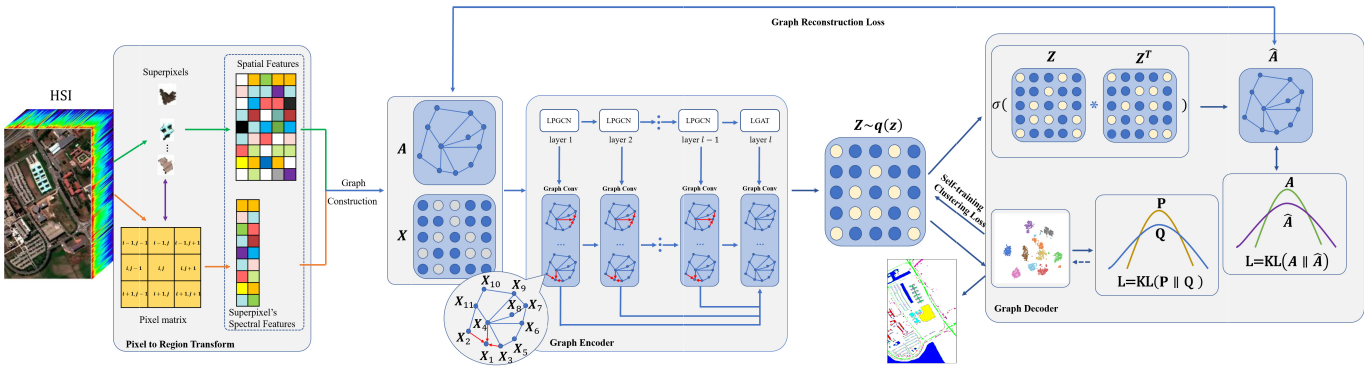


Fig. 1. Flow diagram of the proposed hyperspectral image clustering network with self-supervised locality preserving low-pass graph convolutional embedding (L2GCC). First, the HSI is transformed into graph nodes with the PRT module. Then, the adjacency matrix $\boldsymbol{A}$ is calculated and the graph is constructed. Subsequently, we propose a locality preserving low-pass graph convolutional embedding autoencoder to learn the hidden representation Z, in which the low-pass GCN and layerwise GAT are designed to extract the smoother features and locality features, respectively. Finally, we introduce the self-training clustering to cluster the graph nodes, and the weights are updated by jointly optimizing loss during network training.

Also, the l-order graph convolution is defined as

$$\tilde{\boldsymbol{X}} = (I - k\boldsymbol{L})^l \boldsymbol{X} = \tilde{\boldsymbol{G}}\boldsymbol{X}. \tag{15}$$

In (15), $\tilde{\boldsymbol{G}} = (I - k\boldsymbol{L})^l$ is the filter of l-order graph convolution, and its frequency response function is

$$p(\lambda_q) = (1 - k\lambda_q)^l \tag{16}$$

In practice, the graph Laplacian $\boldsymbol{L}$ is often normalized as

$$\tilde{\boldsymbol{L}}_{\text{sym}} = \tilde{\boldsymbol{D}}^{-\frac{1}{2}} \tilde{\boldsymbol{L}} \tilde{\boldsymbol{D}}^{-\frac{1}{2}} = \tilde{\boldsymbol{U}} \tilde{\boldsymbol{\Lambda}} \tilde{\boldsymbol{U}}^{-1}. \tag{17}$$

Here, $\tilde{\boldsymbol{L}}$ and $\tilde{\boldsymbol{D}}$ are Laplacian matrix and degree matrix corresponding to $\tilde{\boldsymbol{A}} = \boldsymbol{I} + \boldsymbol{A}$ and $\tilde{\boldsymbol{\Lambda}}$ is the decomposition of $\tilde{\boldsymbol{L}}_{\text{sym}}$. Then, we can express the filter as

$$\boldsymbol{G} = I - k\tilde{\boldsymbol{L}}_{\text{sym}}. \tag{18}$$

It should be emphasized that if k is set as 1, the graph filter is the traditional GCN filter. Equation (18) introduces the idea of designing a low-pass convolution kernel, and however, different k values will lead to a different performance of the low-pass filter. Referring to the proof of Theorem 1 and (17), the filtered graph signal $\tilde{\boldsymbol{X}}$ smoothness can be measured by

$$R(\boldsymbol{L}, \tilde{\boldsymbol{X}}) = \frac{\tilde{\boldsymbol{X}}^\top \boldsymbol{L} \tilde{\boldsymbol{X}}}{\tilde{\boldsymbol{X}}^\top \tilde{\boldsymbol{X}}} = \frac{\sum_{i=1}^{n} g_i \lambda_i}{\sum_{i=1}^{n} g_i} \tag{19}$$

where $g_i > 0$ is a coefficient. To obtain a smoother filter, g_i should decrease as λ_i increases. The maximum eigenvalue is denoted by $\lambda_{\max}$. Theoretically, the graph filter is not low-pass, if $k > 1/\lambda_{\max}$, because g_i increases in $(1/k, \lambda_{\max})$. However, not all high-frequency basis signals can be filtered as $k < 1/\lambda_{\max}$. Therefore, the optimal value of k is $1/\lambda_{\max}$.

Referring to the propagation rules of GCN, the propagation function of a low-pass graph convolutional network is formed as

$$\boldsymbol{Z}_{l+1} = \phi\left(\tilde{\boldsymbol{D}}^{\frac{1}{2}} \tilde{\boldsymbol{A}} \tilde{\boldsymbol{D}}^{-\frac{1}{2}} \boldsymbol{Z}_l \boldsymbol{W}^{(l)}\right) \tag{20}$$

where $\boldsymbol{Z}_l$ is the lth layer learned representation and $\boldsymbol{Z}_0 = \boldsymbol{X}$, $\boldsymbol{W}^{(l)}$ is the coefficient matrix learned during the network training.

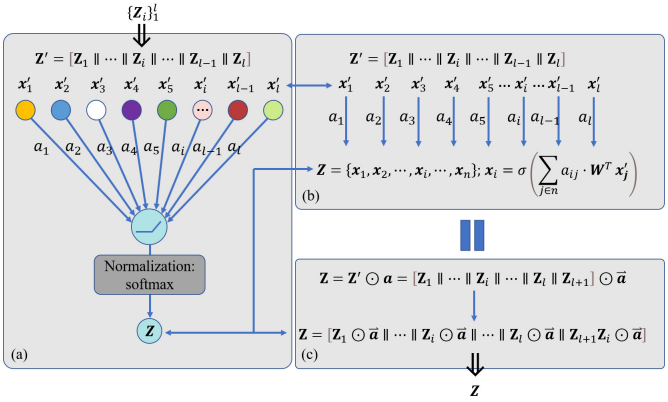


Fig. 2. Conceptual illustrations of the proposed layerwise graph attention (LGAT) module. (a) Visual map of LGAT. (b) Multiscale locality layerwise information is preserved by a concatenation manner, and then, LGAT is illustrated from a nodewise perspective. (c) We express the mechanism of LGAT from a layerwise perspective. Here, the "softmax" is the softmax function for normalization. We adopt "⇓" to present the input and output of LGAT.

2) Layerwise Graph Attention Module (LGAT): As described above, a low-pass graph convolutional filter can be designed by filtering high-frequency basis signals and preserving low-frequency basis signals. However, the low-pass GCN cannot learn layerwise intermediate information for node clustering. In addition, the available deep clustering methods are usually limited to globalization refinement and neglect the multiscale spatial–spectral features embedded in different layers [30]. To address these issues and exploit the multiscale layerwise locality information, we design the LGAT module.

The features at different layers contain different levels of semantic description to depict the input data, and the final clustering task may require different locality layerwise information. The conceptual visual interpretation is shown in Fig. 2.

As different layers contain different spectral dimensions, we preliminarily preserve the multiscale locality layerwise information with a concatenation manner, namely

$$\boldsymbol{Z}' = \left[\boldsymbol{Z}_1 \parallel \cdots \parallel \boldsymbol{Z}_i \parallel \cdots \parallel \boldsymbol{Z}_l \parallel \boldsymbol{Z}_{l+1}\right] \tag{21}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

where l is the graph convolutional layer and $\boldsymbol{Z}_i \in \mathbb{R}^{n\times d_i}$ is the ith layer graph features representations with d_i dimension. In particular, $\boldsymbol{Z}' = \{\boldsymbol{x}_1', \boldsymbol{x}_2', \ldots, \boldsymbol{x}_n'\} \in \mathbb{R}^{n\times d_z}$, where $z = \sum_{i=1}^{l} d_i$ and $\boldsymbol{x}_2'$ is the node feature vector after concatenation operation.

After that, we employ the graph attention mechanism to extract the concatenated features $\boldsymbol{Z}'$, and the attention coefficient a_{ij} between nodes i and j can be formulated as

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp\left(\sigma\left(\boldsymbol{a}^T\left[\boldsymbol{W}\boldsymbol{x}_i' || \boldsymbol{W}\boldsymbol{x}_j'\right]\right)\right)}{\sum_{j\in n}\exp\left(\sigma\left(\boldsymbol{a}^T\left[\boldsymbol{W}\boldsymbol{x}_i' || \boldsymbol{W}\boldsymbol{x}_j'\right]\right)\right)} \tag{22}$$

where $\boldsymbol{a}^T$ is the parameter vector and σ is the activated function, i.e., LeakyReLU(·). Then, the attention-based scale-wise strategy is imposed to (21), and the multiscale locality layerwise features are weighted by the attention coefficient, which is given as follows:

$$\begin{aligned}\boldsymbol{Z} &= \boldsymbol{Z}' \odot \boldsymbol{a} = \left[\boldsymbol{Z}_1 \parallel \cdots \parallel \boldsymbol{Z}_i \parallel \cdots \parallel \boldsymbol{Z}_l \parallel \boldsymbol{Z}_{l+1}\right] \odot \vec{\alpha} \\ &= \left[\boldsymbol{Z}_1 \odot \vec{\alpha} \parallel \cdots \parallel \boldsymbol{Z}_i \odot \vec{\alpha} \parallel \cdots \parallel \boldsymbol{Z}_l \odot \vec{\alpha} \parallel \boldsymbol{Z}_{l+1} \odot \vec{\alpha}\right]\end{aligned} \tag{23}$$

where $\vec{\alpha}$ is the attention coefficient vector learned by (22). Referring to the propagation function of GAT, the graph convolutional form for output features of each node can be calculated as

$$\boldsymbol{x}_i = \sigma\left(\sum_{j\in n}\alpha_{ij}\cdot \boldsymbol{W}^T\boldsymbol{x}_j'\right). \tag{24}$$

From (23), we note that $\boldsymbol{x}_i$ contains rich layerwise information for different clustering tasks.

C. Graph Convolutional Embedding Decoder

In our proposed method, we develop decoders, i.e., graph reconstruction decoder and graph convolutional embedding clustering decoder, to decoder the hidden representation $\boldsymbol{Z}$ learned by locality preserving low-pass graph convolutional embedding encoder and update the trainable weight of L2GCC.

1) Graph Reconstruction Decoder: To reconstruct both node features and the graph structure, a simple inner product decoder is employed to present the reconstructed adjacency matrix in our proposed method, i.e.,

$$\hat{\boldsymbol{A}}_{ij} = \text{sigmoid}\left(\boldsymbol{Z}^\top \boldsymbol{Z}\right) \tag{25}$$

where $\hat{\boldsymbol{A}}_{ij}$ is the adjacency matrix of a reconstructed graph, which is flexible and efficient to represent the structure of the reconstructed graph. In this way, we can decode the content and structure of the original graph. To measure the difference between the reconstructed graph adjacency matrix $\hat{\boldsymbol{A}}$ and the original graph adjacency matrix $\boldsymbol{A}$, we utilize the KL divergence to measure the difference between two distributions. To reduce the influence of individual values in the matrix on the final result, the matrices $\boldsymbol{A}$ and $\hat{\boldsymbol{A}}$ should be normalized first. Then, the graph reconstruction loss is formulated as

$$L_r = \text{KL}\left(\boldsymbol{A} \parallel \hat{\boldsymbol{A}}\right) = \sum_i\sum_j \hat{\boldsymbol{A}}_{ij}\log\frac{\boldsymbol{A}_{ij}}{\hat{\boldsymbol{A}}_{ij}}. \tag{26}$$

2) Graph Convolutional Embedding Self-Training Clustering Decoder: As mentioned above, a graph hidden representation $\boldsymbol{Z}$ can be learned by a locality preserving low-pass graph convolutional embedding autoencoder. However, the clustering does not participate in the autoencoder process, and it is naive to separate clustering from embedding learning, as the learned $\boldsymbol{Z}$ may not suit for clustering. Moreover, the main challenge for unsupervised graph clustering is the nonexistence of labels to train the network. To overcome the problems, inspired by DEC [41], we develop a self-training strategy, in which the self-training clustering objective employs soft labels to supervise the clustering process and obtain appropriate hidden representations for node clustering, and the self-training clustering loss is formulated as

$$L_c = \text{KL}(P \parallel Q) = \sum_i\sum_j p_{ij}\log\frac{p_{ij}}{q_{ij}} \tag{27}$$

where $\text{KL}(\cdot\|\cdot)$ is the KL divergence, p_{ij} is the target distribution, and Q is soft labels distribution. We can adopt q_{ij} to measure the similarity between cluster center μ_j and node embedding z_i of node i, which is measured by a student's t-distribution, namely

$$q_{ij} = \frac{\left(1+\left\|z_i-\mu_j\right\|^2\right)^{-1}}{\sum_j'\left(1+\left\|z_i-\mu_{j'}\right\|^2\right)^{-1}} \tag{28}$$

and we can treat each node as a soft clustering assignment distribution. Also, we define the target distribution p_{ij} as

$$p_{ij} = \frac{q_{ij}^2/\sum_i q_{ij}}{\sum_k\left(q_{ik}^2/\sum_i q_{ik}\right)} \tag{29}$$

where $\sum_i q_{ij}$ is a normalization to normalize each centroid's loss contribution and prevent hidden feature space from being distorted by large clusters.

As described above, a denser Q distribution can be given by minimizing L_c. Namely, it makes the distribution of nodes in the class more similar, and the distance between nodes in different classes is longer. In this way, we can transform unsupervised clustering into "semisupervised" clustering.

D. Joint Embedding and Optimization

In our method, the L2GCC is optimized by integrating the graph embedded autoencoder and self-training clustering, and then, we can define the total objective function as

$$L = L_r + \gamma L_c \tag{30}$$

where L_r denotes the reconstruction loss of GAE in (26), L_c is the self-training clustering loss formulated in (27), and $\gamma \geq 0$ is a coefficient weight that is used to control embedded space degree distorting and set as 0.1 in our method.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

We should update three paraments by adopting gradient descent in the L2GCC, including the coefficient weight $W^{(l)}$, cluster center μ_j, and target distribution p_{ij}.

$$\frac{\partial L_c}{\partial \mu_j} = 2\sum_{i=1}^{N}\left(1+\|z_i-\mu_j\|^2\right)^{-1}\left(q_{ij}-p_{ij}\right)\left(z_i-\mu_j\right). \quad (31)$$

1) Weights and Cluster Centers Updating: The gradient of L_c relative to μ_j with the fixed p_{ij} can be expressed as

Given a learning rate, we can update μ_j by

$$\mu_j = \mu_j - \eta\frac{\partial L_c}{\partial \mu_j}. \quad (32)$$

Then, the coefficient weight $W^{(l)}$ is updated by

$$W^{(l)} = W^{(l)} - \eta\left(\frac{\partial L_r}{\partial W^{(l)}} + \gamma\frac{\partial L_c}{\partial W^{(l)}}\right). \quad (33)$$

2) Target Distribution Updating: The target distribution p_{ij} can be treated as "ground-truth labels" during the training process and should be updated in every T iteration using all embedded nodes. The m initial centroids $\{\mu_j\}_{j=1}^{m}$ are obtained by conducting k-means clustering on the embedded nodes $\boldsymbol{Z}$. Then, p_{ij} is updated by (28) and (29), and the label of n_i could be estimated by

$$n_i = \arg\max_j q_{ij}. \quad (34)$$

To this end, the L2GCC autoencoder is first trained without self-training clustering to learn a well-presentation $\boldsymbol{Z}$ as demonstrated in Section III-B. Then, we perform self-training clustering to improve the embedding. The training progress will be stopped as the target distribution p_{ij} change between two consecutive updates is less than a fixed δ or the training epochs T reach the upper set limit. The clustering result is presented by the last optimized soft labels distribution Q. The training process of the proposed L2GCC is shown in Algorithm 1.

Algorithm 1 L2GCC Training Process

Input: Graph G; Number of initial centroids m; Number of iterations T; Coefficient weight γ; Distribution threshold δ.
Output: Clustering results Q.

Initialization: $t=1$; $\boldsymbol{Z}^{(0)}=\boldsymbol{X}$; Initialize the parameters of graph autoencoder;
Calculate graph Laplacian $\tilde{\boldsymbol{L}}_{sym}$ by Eq. (17);
$k \leftarrow 1/\lambda_{max}$;
Get the graph hidden presentation $\boldsymbol{Z}$ by performing graph autoencoder according to minimizing Eq. (26);
Get m initial clusters centroid μ_j by k-means clustering based on $\boldsymbol{Z}$.
for $t=1$ *to* $T-1$ **do**
  The soft labels distribution Q is calculated based on μ and $\boldsymbol{Z}$ by Eq. (28);
  The target distribution P is calculated based on Q by Eq. (29);
  The self-training clustering loss L_c is calculated by Eq. (27);
  if $L_c \le \delta$ **break**;
  Calculate joint embedding loss L by Eq. (30) and update the weight matrices adopting Adam gradient descent;
end
Obtain the final clustering results Q according to Eq. (34).

E. HSI Preprocessing and Graph Construction

HSI contains rich spectral information and a large number of pixels, which not only provides sufficient features for target classification but also brings a huge amount of calculation. In practice, the amount of calculation complexity of graph convolutional neural network (GNN) increases exponentially with the increase of the number of nodes. It is unrealistic to calculate each pixel as a node of the graph. In other words, how to reduce the number of nodes while preserving the hyperspectral spectral–spatial characteristics is a major problem in HSI preprocessing. To construct the graph that can then be clustered by the subsequent network, we introduce an HSI preproposing method, which contains two parts, including spatial module and spectral module.

1) *Spatial Module:* Given an HSI cube $I_B = \{x_1, x_2, \ldots, x_m\}$ with $m = \mathcal{W}\times\mathcal{H}$ pixels and $\mathcal{B}$ spectral bands, where $\mathcal{W}$ and $\mathcal{H}$ denote the width and height, respectively, first, we adopt unsupervised PCA [44] to reduce the dimensionality of HSI and the first principal component is preserved to generate a reduced image $I_b = \{x_1^r, x_2^r, \ldots, x_m^r\}$. Then, we utilize the SLIC [45] to assign the pixels into superpixels, and the mathematical expression of the local superpixel can be defined as

$$\mathrm{HSI} = \cup_{i=1}^{K}\mathcal{S}_i,\ \mathcal{S}_i\cap\mathcal{S}_j=\emptyset,\quad i\neq j; i,\ j=1,2,\ldots,K \quad (35)$$

where $\mathcal{S}_i = \{p_{i,1},\ldots,p_{i,n_i}\}$ denotes a superpixel containing n_i connected pixels and K is the total number of superpixels.

2) *Spectral Module:* As shown above, we can partition the HSI into superpixels by the spatial module. However, the dimensionality reduction loses the spectral information of HSI, and namely, the superpixel does not fully extract the spectral features of HSI. To solve the problem, we extract the spectral value of each pixel from the original HSI directly, and then, we calculate the mean spectral value of pixels in superpixel as the superpixel's spectral features. We can express the spectral feature vector of the pixel at the spatial location $p_0=(x,y)$ as

$$\boldsymbol{x}(\boldsymbol{p}_0) = \left(X_1(\boldsymbol{p}_0), X_2(\boldsymbol{p}_0), \ldots, X_B(\boldsymbol{p}_0)\right). \quad (36)$$

where $X_i(\boldsymbol{p}_0)$ denotes the spectral value of the pixel in the ith channel at the spatial location $\boldsymbol{p}_0$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

Then, the mathematical expression of the HSI feature $\boldsymbol{X}$ is

$$\begin{aligned}\boldsymbol{X} &= [\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots \boldsymbol{X}_S]^T \\ &= \left[\frac{1}{\tau_1}\sum_{k=1}^{N_1}\boldsymbol{x}_k^1, \frac{1}{\tau_2}\sum_{k=1}^{N_2}\boldsymbol{x}_k^2, \ldots, \frac{1}{\tau_K}\sum_{k=1}^{N_S}\boldsymbol{x}_k^S\right]^T \end{aligned} \tag{37}$$

where $\boldsymbol{X}_i$ is the ith superpixel feature vector, τ_i denotes the number of pixels contained in superpixel, and $\boldsymbol{x}_k^i$ is the spectral feature vector of the pixel shown as (36).

3) *Graph Construction:* In the proposed method, we take the superpixels as the graph node, and the adjacency matrix $\boldsymbol{A}$, defining the relationships between vertexes, can be generally computed as a radial basis function (RBF)

$$\boldsymbol{A}_{ij} = \begin{cases} e^{-\rho\|\mathbf{x}_i - \mathbf{x}_j\|^2}, & \text{if } \boldsymbol{X}_i \in N_t\left(\boldsymbol{X}_j\right) \text{ or } \boldsymbol{X}_j \in N_t(\boldsymbol{X}_i) \\ 0, & \text{otherwise} \end{cases} \tag{38}$$

where $\rho = 0.2$ is an empirical set to control the RBF's width, $\boldsymbol{X}_i$ denotes node features of nodes i, and $\mathrm{N}_t(\boldsymbol{X}_i)$ is the t-hop neighbors of $\boldsymbol{X}_i$.

F. Computational Complexity

Let n be the number of graph nodes, d be the input node spectral dimension, d_i be the ith layer node spectral dimension, $|\mathcal{E}|$ be the number of nonzero entries of the adjacency matrix A, c be the number of clusters, and l be the number of graphs embedded autoencoder layers. The computational complexity of low-pass GCN convolution is calculated as $O[(n + |\mathcal{E}|)(dd_1 + \sum_{i=2}^{l-1} d_i d_{i-1})]$. For layerwise GAT, the computational complexity is $O[|\mathcal{E}|(\sum_{i=1}^{l-1} d_i)d_l]$. The computational complexity of graph autoencoder is $O[(n + |\mathcal{E}|)(dd_1 + \sum_{i=2}^{l-1} d_i d_{i-1}) + |\mathcal{E}|(\sum_{i=1}^{l-1} d_i)d_l]$. For (29), the computational complexity is $O[(nc + n\log n) + (n^2 + n\log n)]$. Therefore, the computational complexity of performing clustering on the graph in each iteration is $O[(n + |\mathcal{E}|)(dd_1 + \sum_{i=2}^{l-1} d_i d_{i-1}) + |\mathcal{E}|(\sum_{i=1}^{l-1} d_i)d_l + (nc + n\log n) + (n^2 + n\log n)]$. If our model iterates T times, we express the overall computational complexity of LGCC is $O[[(n + |\mathcal{E}|)(dd_1 + \sum_{i=2}^{l-1} d_i d_{i-1}) + |E|(\sum_{i=1}^{l-1} d_i)d_l + (nc + n\log n) + (n^2 + n\log n)]T]$. From the expression formula of computational complexity, we can greatly reduce the computational complexity by reducing the number of nodes, which verifies the effectiveness of the HSI preprocessing.

IV. Proofs

The smoother signals can be obtained by preserving low-frequency basis signals and filtering high-frequency basis signals.

Proof: Given the graph signal (node features), considering $\boldsymbol{L} = \boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{-1}$ and referring to (10), we can express the eigenvector $\boldsymbol{u}_q$ smoothness as

$$R\left(\boldsymbol{L}, \boldsymbol{u}_q\right) = \frac{\boldsymbol{u}_q^\top \boldsymbol{L}\boldsymbol{u}_q}{\boldsymbol{u}_q^\top \boldsymbol{u}_q} = \lambda_q \tag{39}$$

where $\boldsymbol{u}_q(i)$ is the $\boldsymbol{u}_q$'s ith element. Equation (39) shows that smoother eigenvectors have smaller eigenvalues. Then, we can decompose $\boldsymbol{X}$ into a linear combination, i.e.,

$$\boldsymbol{X} = U\boldsymbol{c} = \sum_{q=1}^{n} c_q \boldsymbol{u}_q \tag{40}$$

where c_q denotes the coefficient of the qth eigenvector $\boldsymbol{u}_q$. Thus, the smoothness of $\boldsymbol{X}$ can be calculated based on (10), (39), and (40)

$$R(\boldsymbol{L}, \boldsymbol{X}) = \frac{\mathbf{X}^\top \boldsymbol{L}\boldsymbol{X}}{\boldsymbol{X}^\top \boldsymbol{X}} = \frac{\sum_{c=1}^{n} c_q^2 \lambda_c}{\sum_{c=1}^{n} c_q^2}. \tag{41}$$

Therefore, Theorem 1 is proven.

Theorem 2: Given a graph filter $\boldsymbol{G}$, if its frequency response function $p(\lambda)$ is nonnegative and nonincreasing for all λ_i, then given any signal $\boldsymbol{X}$ and the filtered $\tilde{\boldsymbol{X}} = \boldsymbol{G}\boldsymbol{X}$, we always have

$$R\left(\tilde{\boldsymbol{X}}, \boldsymbol{X}\right) \le R(\boldsymbol{L}, \boldsymbol{X}). \tag{42}$$

Namely

$$\frac{\sum_{i=1}^{n} b_i \lambda_i}{\sum_{i=1}^{n} b_i} \le \frac{\sum_{i=1}^{n} c_i \lambda_i}{\sum_{i=1}^{n} c_i}. \tag{43}$$

Proof: We adopt induction to prove Theorem 2, namely

$$\begin{aligned} R_b^i &= \frac{\sum_{i=1}^{n} b_i \lambda_i}{\sum_{i=1}^{n} b_i} \\ R_c^i &= \frac{\sum_{i=1}^{n} c_i \lambda_i}{\sum_{i=1}^{n} c_i}. \end{aligned} \tag{44}$$

If $\lambda_1 \le \cdots \lambda_n$ and $(c_1/b_1) \le \cdots \le (c_n b_n$, we can easily prove that $R_b^2 \le R_c^2$. Now, suppose that $n = l - 1$ and $R_b^{l-1} \le R_c^{l-1}$, and then, we have R_b^l

$$\begin{aligned} \frac{\sum_{i=1}^{l} b_i \lambda_i}{\sum_{i=1}^{l} b_i} &= \frac{\sum_{i=1}^{l-1} b_i \lambda_i + b_l \lambda_l}{\sum_{i=1}^{l-1} b_i + b_l} \\ &= \frac{\left(\sum_{i=1}^{l-1} b_i\right)\lambda_b^{(l-1)} + b_l \lambda_l}{\sum_{i=1}^{l-1} b_i + b_l} \\ &\le \frac{\left(\sum_{i=1}^{l-1} b_i\right)\lambda_c^{(l-1)} + b\lambda_l}{\sum_{i=1}^{l-1} b_i + b_l}. \end{aligned} \tag{45}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DETAILS OF THE DATASETS USED FOR THE EVALUATION OF THE INVESTIGATED ALGORITHMS

NO.	Indian Pines		Salinas		UH2013	
	Landcover name	Pixels	Landcover name	Pixels	Landcover name	Pixels
1	Asphalt	46	Weed 1	2009	Healthy-grass	1251
2	Corn-notill	1428	Weed 2	3726	Stressed-grass	1254
3	Corn-mintill	830	Fallow	1976	Synthetic-grass	697
4	Corn	237	Fallow-plow	1394	Tress	1244
5	Grass-pasture	483	Fallow-smooth	2678	Soil	1242
6	Grass-trees	730	Stubble	3959	Water	325
7	Grass-pasture-mowed	28	Celery	3579	Residential	1268
8	Hay-windrowed	478	Grapes untrained	11271	Commercial	1244
9	Oats	20	Soil	6203	Road	1252
10	Soybean-notill	972	Corn	3278	Highway	1227
11	Soybean-mintill	2455	Lettuce-4wk	1068	Railway	1235
12	Soybean-clean	593	Lettuce-5wk	1927	Parking-Lot 1	1233
13	Wheat	205	Lettuce-6wk	916	Parking-Lot 2	469
14	Woods	1265	Lettuce-7wk	1070	Tennis-Court	428
15	Buildings-Grass-Trees-Drives	386	Vineyard-untrained	7268	Running-Track	660
16	Stone-Steel-Towers	93	Vineyard-trellis	1807		

Since $R_b^{l-1} \leq R_c^{l-1}$ and $((\sum_{i=1}^{l-1} c_i)/(\sum_{i=1}^{l-1} b_i)) \leq (c_l/b_l)$, we have

$$\frac{\left(\sum_{i=1}^{l-1} b_i\right)\lambda_c^{(l-1)} + b\lambda_l}{\sum_{i=1}^{l-1} b_i + b_l} \leq \frac{\left(\sum_{i=1}^{l-1} c_i\right)\lambda_c^{(l-1)} + c\lambda_l}{\sum_{i=1}^{l-1} c_i + c_l} = \frac{\sum_{i=1}^{l} c_i\lambda_i}{\sum_{i=1}^{l} c_i}. \tag{46}$$

Hence, Theorem 2 holds for all n.

V. EXPERIMENTS

A. Datasets

In the experiments, we employ three widely used HSI datasets to test the performances of the investigated methods, including Indian Pines (IP), Salinas, and University of Houston 2013 (UH2013) datasets. The IP dataset contains 200 spectral bands with an image size of 145 × 145 and 16 land-cover classes. The Salinas dataset is composed of 204 spectral channels with a size of 610 × 340 and nine land-cover classes. For the Houston 2013 dataset, the data size is 349 × 1905 with 144 channels. The image is labeled in 15 categories. The details of the three datasets used in the experiment are shown in Table I.

B. Baselines and Evaluation Metrics

To verify the clustering accuracy and real-time performances of the proposed L2GCC, we consider ten baselines for comparison, including k-means [15], FCM [17], FCM-S1 [22], SC [16], SSSC [24], S^5C [25], DS^3C-Net [11], AE [46] + k-means, DEC [30], and CC [34]. Notably, k-means, FCM, FCM-S1, and SC are traditional clustering methods designed for natural images, and SC and CC are spectral clustering method and contrastive clustering method, respectively. SSSC and S^5C belong to SSC methods. Furthermore, DS^3C-Net, AE+k-means, DEC, and CC are deep clustering methods and are developed for large-scale HSI clustering. The hyperparameters of the aforementioned methods are the same as their original papers.

TABLE II
OPTIMAL VALUES UTILIZED IN L2GCC

Dataset	S	L	T	η	c	γ	δ
IP	1000	3	200	0.001	16	0.1	0.1%
Salinas	7000	3	200	0.001	16	0.1	0.1%
UH 2013	14000	3	200	0.001	15	0.1	0.1%

In our experiments, five metrics, including overall accuracy (OA), per-class accuracy (PA), kappa coefficient (κ), normalized mutual information (NMI), and adjusted Rand index (ARI), are introduced for all the investigated methods to estimate their performances.

C. Experimental Settings

In our method, there are mainly six parameters, including the number of superpixels S (graph node), the number of low-pass graph convolutional layers L, learning rate η, the number of epochs T, the number of initial centroids c, coefficient weight γ, and distribution threshold δ, should be set, and their optimal values utilized in the proposed method are shown in Table II. It should be noted that the setting of δ is very small and will not be triggered under normal circumstances, except for the discrete case of clustering. The c initial centroids $\{\mu_j\}_{j=1}^{c}$ are obtained by conducting k-means clustering, and its value is equal to the number of land covers contained in the HSI. The dimensions of the three hidden layers are 168, 84, and 42.

The experiments in this article are implemented on the Intel i9-10900K processor with 3.70 GB of DDR4 RAM. Regarding the graphic processing unit (GPU), the NVIDIA GeForce GTX 1080Ti with 11 GB of DDR4 RAM is used.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

TABLE III
QUANTITATIVE EXPERIMENTAL PERFORMANCES OF INVESTIGATED METHODS ON IP, AND BOLD DENOTES THE BEST PERFORMANCES

No.	k-means	FCM	FCM-S1	SC	SSSC	S^5C	DS^3C-Net	AE+ k-means	DEC	CC	L2GCC
1	0.0594	0.0567	0.0634	0.0000	0.0236	0.1125	0.1142	0.0870	0.0000	0.0870	**0.2543**
2	0.4563	0.5226	0.4426	0.0273	0.1836	0.2537	0.2709	0.1408	0.1877	0.2955	**0.6787**
3	0.1819	0.2118	0.1781	0.0048	0.3216	0.3576	**0.6985**	0.1940	0.2892	0.5795	**0.6028**
4	0.1237	0.1058	0.2109	0.0000	0.3018	0.1531	0.8861	0.1730	0.2405	0.8861	**0.9017**
5	0.3447	0.2946	0.3393	0.0166	0.6825	0.6279	**0.6534**	0.5217	0.0000	0.6584	**0.8236**
6	0.6901	0.7645	0.8008	0.0000	0.4913	0.5524	0.9795	0.9329	0.9726	0.9795	**1.0000**
7	0.0000	0.1043	0.0000	0.0000	0.6127	**0.7019**	0.0000	0.0000	0.0000	0.0000	0.0000
8	0.6239	0.8922	0.5706	**1.0000**	0.5736	0.5124	0.9137	0.7134	0.6464	**1.0000**	**1.0000**
9	0.0000	**0.6400**	0.0215	0.0000	0.0000	0.1329	0.0000	0.0000	0.0000	0.0000	0.3285
10	0.5654	0.2528	0.5268	0.0031	0.3162	0.2618	0.5327	0.5021	0.3693	0.4095	**0.7021**
11	0.4733	0.5141	0.5966	**0.9609**	0.3769	0.2235	0.2957	0.3752	0.3202	0.2957	0.6833
12	0.2592	0.2788	0.2553	0.0017	0.1823	0.0000	0.6572	0.1551	0.2749	**0.7437**	0.4564
13	0.3276	0.4289	0.2956	0.1463	0.6571	0.7531	**1.0000**	0.5024	0.3610	**1.0000**	**1.0000**
14	0.8559	0.8006	0.9173	**1.0000**	0.4839	0.4629	0.6854	0.7605	0.7763	0.6854	0.8346
15	0.2007	0.2869	0.2584	0.0000	0.2649	0.0000	**1.0000**	0.6995	0.7021	**1.0000**	**1.0000**
16	0.0000	0.0632	0.0000	0.7419	0.68310	0.7726	0.1986	0.5591	0.6022	0.0000	**0.8235**
OA	0.3811	0.3757	0.3589	0.4153	0.3828	0.4072	0.5738	0.4458	0.4171	0.5514	**0.7351**
κ	0.3572	0.2975	0.2782	0.2794	0.3326	0.3518	0.5364	0.3847	0.3591	0.5192	**0.6829**
NMI	0.4386	0.4380	0.4217	0.5041	0.4062	0.4317	0.6928	0.4527	0.4787	0.6644	**0.7516**
ARI	0.2195	0.2295	0.2070	0.2455	0.2138	0.2296	0.4350	0.2661	0.2968	0.4070	**0.5326**

TABLE IV
QUANTITATIVE EXPERIMENTAL PERFORMANCES OF INVESTIGATED METHODS ON SALINAS, AND BOLD DENOTES THE BEST PERFORMANCES. "OOM" DENOTES THE OUT OF MEMORY

No.	k-means	FCM	FCM-S1	SC	SSSC	S^5C	DS^3C-Net	AE+ k-means	DEC	CC	L2GCC
1	0.9985	0.4179	0.4860	OOM	0.0416	0.0000	0.8761	0.0000	0.9000	**1.0000**	**1.0000**
2	0.5698	0.5524	0.5637	OOM	0.9875	0.9628	0.9473	**1.0000**	0.7933	0.8969	0.9826
3	0.9741	0.8501	**1.0000**	OOM	0.0154	0.0434	**1.0000**	0.5501	0.0000	**1.0000**	0.9234
4	0.9865	0.0000	0.9537	OOM	0.8131	0.8512	0.6502	0.9340	0.5079	0.0000	0.8762
5	**0.7856**	0.5540	0.7365	OOM	0.8723	0.7526	**0.8490**	0.7367	0.7412	0.5310	0.7731
6	0.9952	0.9990	0.9704	OOM	0.9358	0.9832	0.7614	0.8406	0.8366	0.6603	**1.0000**
7	0.4810	0.0000	0.0003	OOM	0.5412	0.1126	0.5236	0.3328	0.6943	0.7499	0.7806
8	0.7309	0.7772	0.8412	OOM	0.4226	0.4917	0.4608	0.5008	**0.8565**	0.3615	0.8219
9	0.9750	0.6969	0.9602	OOM	0.5638	0.6826	0.6602	**0.9997**	0.9994	0.7201	0.9239
10	0.6169	0.5507	0.5429	OOM	0.1143	0.3158	0.8133	0.5863	0.3350	0.9451	**0.9137**
11	0.3370	0.0000	0.3013	OOM	0.6915	0.6213	**0.8315**	0.0000	0.0000	**1.0000**	0.7129
12	0.0273	0.0047	0.0000	OOM	0.4857	0.5184	**1.0000**	0.9460	0.8293	**0.9984**	0.9032
13	0.3548	0.8134	0.4613	OOM	0.0000	**0.9321**	0.0000	0.0000	0.0000	0.0000	0.6513
14	0.5584	0.6512	0.7469	OOM	0.1826	**0.8725**	0.0000	0.9645	0.0000	0.0000	0.6228
15	0.4994	0.4279	0.5064	OOM	0.4682	0.3948	0.5687	0.6237	0.0579	0.5687	**0.8027**
16	0.0000	0.8321	0.8433	OOM	0.9265	0.6617	**0.9182**	0.9917	0.9092	**1.0000**	0.8245
OA	0.6799	0.5673	0.6318	OOM	0.5243	0.5168	0.6673	0.6568	0.6257	0.6396	**0.8315**
κ	0.6572	0.5431	0.6202	OOM	0.5069	0.5035	0.6326	0.6206	0.5796	0.6123	**0.8226**
NMI	0.7356	0.6859	0.7089	OOM	0.6423	0.6327	0.7785	0.7720	0.7705	0.7806	**0.8569**
ARI	0.5466	0.4693	0.5119	OOM	0.4676	0.4359	0.5531	0.5548	0.5824	0.5330	**0.6926**

To minimize the random influence, we execute all experiments ten times and report the average clustering performances with Pytorch-Geometric as a backend.

D. Experimental Performances Comparison

In this section, the quantitative clustering results on the IP, Salinas, and UH2013 datasets are reported in detail in Tables III–V, respectively. The best performances and the second-best performances are marked in bold and underlined, respectively. The corresponding visual clustering results are shown in Figs. 3–5.

1) *Quantitative Results for the IP Dataset:* From the quantitative clustering results in Table III, the following conclusions can be easily summarized. First, L2GCC achieves the best results among all key indicators in the experimental results, which shows that our model has a good filtering ability for interference noise. Specifically, compared with the second-best results, the results of our proposed model improved by 18.37%, 16.37%, 8.72%, and 12.56% in OA, κ, NMI, and ARI, respectively. Second, compared with classical methods, i.e., k-means, FCM, and FCM-S1, scalable spectral/subspace clustering methods can achieve superior performances since their mechanism design can help to feature extraction. Specifically, SC, SSSC, and S^5C have higher clustering accuracies, and the OAs of the three methods are improved by 3.42%, 0.17%, and 2.61%, respectively,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

TABLE V

QUANTITATIVE EXPERIMENTAL PERFORMANCES OF INVESTIGATED METHODS ON UH2013, AND BOLD DENOTES THE BEST PERFORMANCES. "OOM" DENOTES OUT OF MEMORY

No.	k-means	FCM	FCM-S1	SC	SSSC	S^5C	DS^3C-Net	AE+ k-means	DEC	CC	LGCC
1	0.7926	0.4932	0.4762	OOM	0.5238	0.4813	**0.9235**	0.8129	0.8185	0.7506	**0.9017**
2	0.1436	0.2831	0.3017	OOM	0.3671	0.2516	0.8702	0.0973	**0.7783**	0.6938	0.4826
3	0.7638	0.0000	0.0000	OOM	0.0000	0.0000	0.0000	0.9842	0.4390	0.9900	**0.9999**
4	0.7024	0.3834	0.3921	OOM	0.4229	0.4420	0.8135	0.7436	0.0000	0.6624	**0.8731**
5	0.0000	0.6932	0.6273	OOM	0.2783	0.0000	0.7761	0.8285	**0.9815**	0.7198	0.7531
6	0.0000	0.0185	0.0372	OOM	0.1541	0.1266	0.2118	**0.7600**	0.0000	0.0000	0.6128
7	0.2416	0.0789	0.0564	OOM	0.0000	0.0510	0.3970	**0.8028**	0.0000	0.6088	0.6529
8	0.0000	0.0949	0.0629	OOM	0.2576	0.1932	0.6290	0.2291	0.2098	0.3778	**0.5326**
9	0.2671	0.1070	0.1324	OOM	0.0962	0.1224	0.4702	0.0000	**0.4744**	0.3011	0.0932
10	0.2935	0.2347	0.2179	OOM	0.2967	0.3125	0.5213	0.2771	**0.6544**	0.3325	0.5932
11	0.2613	**0.5417**	0.4386	OOM	0.3629	0.2718	0.3926	0.2777	0.0089	0.2874	0.4217
12	0.7132	0.8021	0.8211	OOM	0.7941	0.8530	0.6211	0.7875	0.5929	0.4039	**0.8871**
13	0.0000	0.0000	0.000	OOM	0.0000	0.0000	0.4425	0.0000	**0.6482**	0.6077	0.6168
14	**1.0000**	0.0000	0.0000	OOM	0.0000	0.0000	0.0000	0.9907	**1.0000**	0.0000	0.5132
15	0.3818	0.0000	0.0427	OOM	0.0000	0.0000	0.5233	0.5727	0.6258	0.9924	**1.0000**
OA	0.3976	0.3070	0.3182	OOM	0.3352	0.3167	0.5473	0.5180	0.4704	0.5348	**0.6412**
κ	0.3641	0.2456	0.2573	OOM	0.3219	0.3221	0.5126	0.4783	0.4278	0.5008	**0.6315**
NMI	0.5217	0.3623	0.3716	OOM	0.4153	0.4078	0.6094	0.5871	0.5660	0.5949	**0.6931**
ARI	0.2638	0.1488	0.1534	OOM	0.2035	0.1726	0.4103	0.3613	0.3330	0.3977	**0.5228**

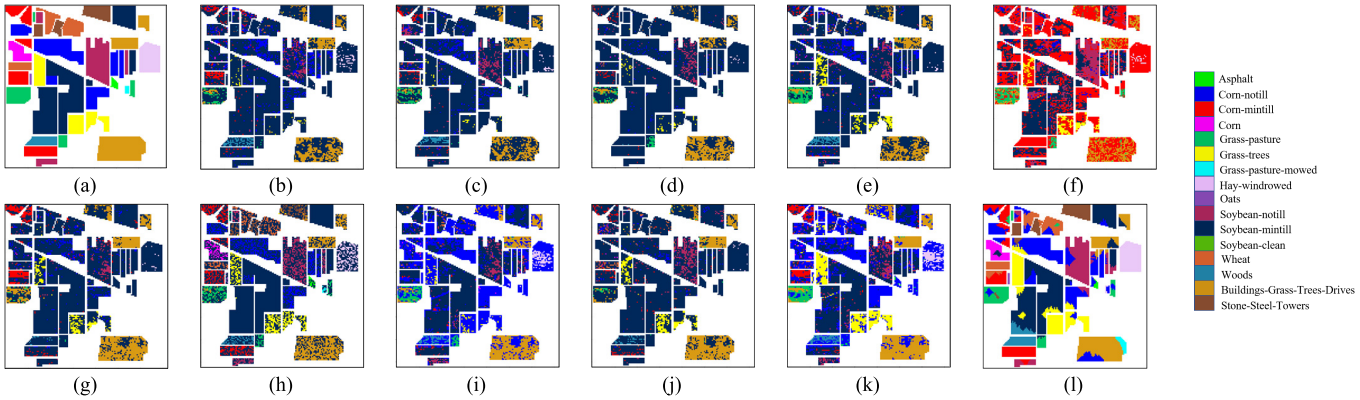


Fig. 3. Visual clustering map of investigated methods on the IP dataset: (a) ground-truth map, (b) k-means (OA = 38.11%), (c) FCM (OA = 37.57%), (d) FCM-S1(OA = 35.89%), (e) SC (OA = 41.53%), (f) SSSC (OA = 38.28%), (g)S^5C (OA = 40.72%), (h) DS^3C-Net (OA = 57.38%), (i) AE+k-means (OA = 44.58%), (j) DEC (OA = 41.71%), (k) CC (OA = 55.14%), and (l) L2GCC (OA = 73.51%).

compared with k-means. The deep clustering algorithms (i.e., DS^3C-Net, AE +k-means, DEC, CC, and L2GCC) are significantly better than shallow methods (i.e., k-means, FCM, FCM-S1, and SC), mainly because deep learning can extract high-level features for clustering. Specifically, compared with the best results in shallow methods, the clustering accuracies of deep clustering algorithms in terms of the OA are increased by 3.77%, 3.05%, 0.18%, and 13.16%. It should also be noted that the graph-based clustering method DS^3C-Net obtains competitive results among deep clustering methods since it can extract the complex nonlinear similarities among pixels by converting high-dimensional spectral features into a subspace.

2) *Quantitative Results for the Salinas Dataset:* From Table IV, the following conclusions can be obtained. First, k-means achieves the best results in classical methods, which shows that k-means can successfully identify more categories than the other two methods. Specifically, the OA of k-means is 38.11%, which is improved by 0.54% and 2.22% compared with the value of FCM (37.57%) and FCM-S1(35.89%). Second, for the spectral/subspace clustering methods, SSSC achieves more effective performance, its OA (52.43%), κ (50.69%), NMI (64.23%), and ARI (46.76%) are improved by 0.75%, 0.34%, 0.96%, and 3.17%, respectively, compared with these values of S^5C. Furthermore, the performances of SSSC and S^5C are better than the classical methods. However, SC failed to handle the clustering because of out-of-memory, which shows that SC needs to be improved in the clustering adaptability of large datasets. Third, for the quantitative results obtained by deep and shallow clustering methods, different from our consideration, deep clustering methods do not achieve the expected clustering advantages, which illustrates that deep clustering methods are not always superior to

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

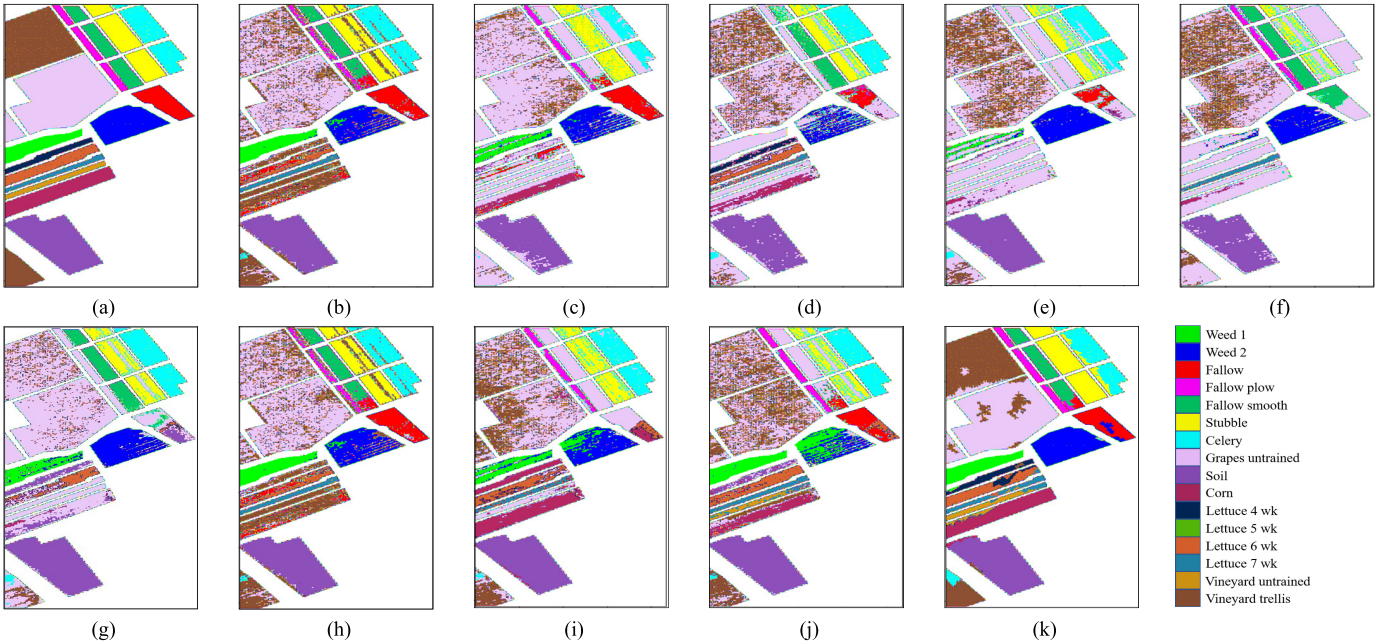


Fig. 4. Visual clustering map of investigated methods on the Salinas dataset: (a) Ground-truth map, (b) k-means (OA = 67.99%), (c) FCM (OA = 56.73%), (d) FCM-S1(OA = 63.18%), (e) SSSC(OA = 52.43%), (f) S^5C(OA = 51.68%), (g) DS^3C-Net (OA = 66.73%), (h) AE+ k-means (OA = 65.68%), (i) DEC (OA = 62.57%), (j) CC (OA = 63.96%), and (k) L2GCC (OA = 83.15%).

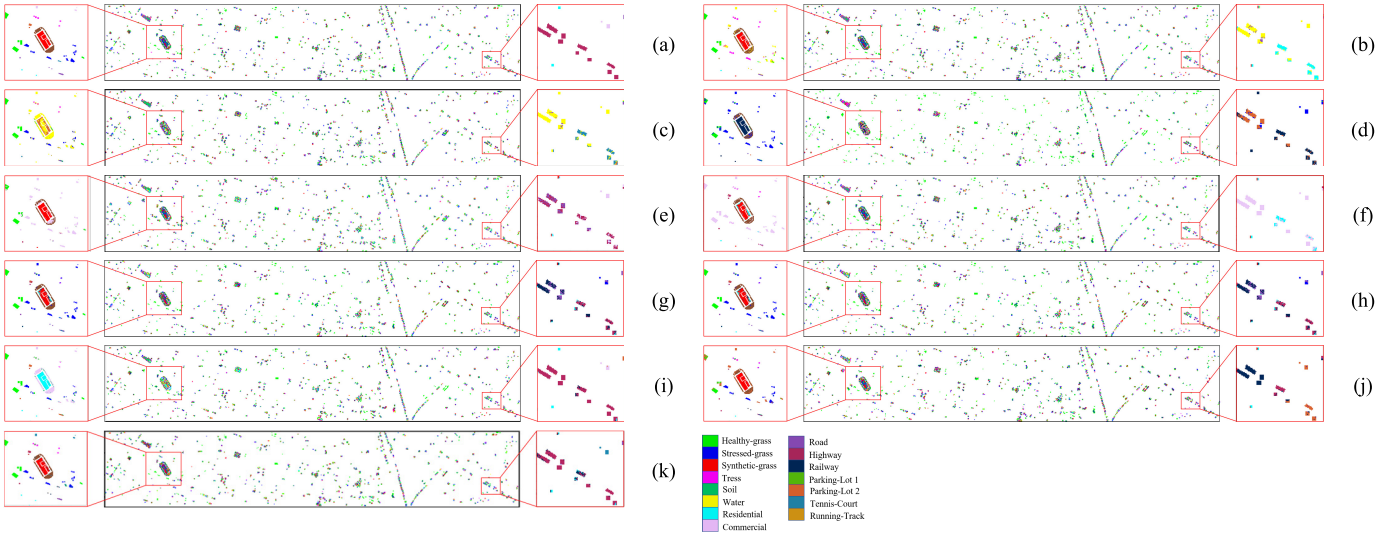


Fig. 5. Visual clustering map of investigated methods on the UH2013 dataset: (a) ground-truth map, (b) k-means (OA = 39.76%), (c) FCM (OA = 30.70%), (d) FCM-S1(OA = 31.82%), (e) SSSC(OA = 33.52%), (f) S^5C(OA = 31.67%), (g) DS^3C-Net (OA = 54.73%), (h) AE+ k-means (OA = 51.80%), (i) DEC (OA = 47.04%), (j) CC (OA = 53.48%), and (k) L2GCC (OA = 64.12%).

shallow clustering methods. However, DS^3C-Net obtains more competitive performance and improves OA by 1.05%, 4.16%, and 2.77% in contrast to the value of AE+k-means, DEC, and CC, respectively. Fourth, the proposed L2GCC achieves the best clustering results by improving the OA, κ, NMI, and ARI by 13.74%, 12.09%, 5.84%, and 10.04%, respectively, compared with this value of the second-best method, which signifies the effectiveness of the proposed method.

3) *Quantitative Results for the UH2013 Dataset:* From the quantitative clustering results in Table V, the following conclusions can be easily summarized. First, compared with the clustering results of IP and Salinas datasets, the clustering results of all investigated methods in UH2013 are greatly decreased. The main reason is that the UH2013 is larger and contains more pixels and complex interrelationships, which poses higher challenges to the investigated algorithms, and SC is suffering from out-of-memory. Second, different from the results in Salinas, deep clustering methods achieve superior results to shallow methods, which indicates the good adaptability to large-scale dataset clustering of deep

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

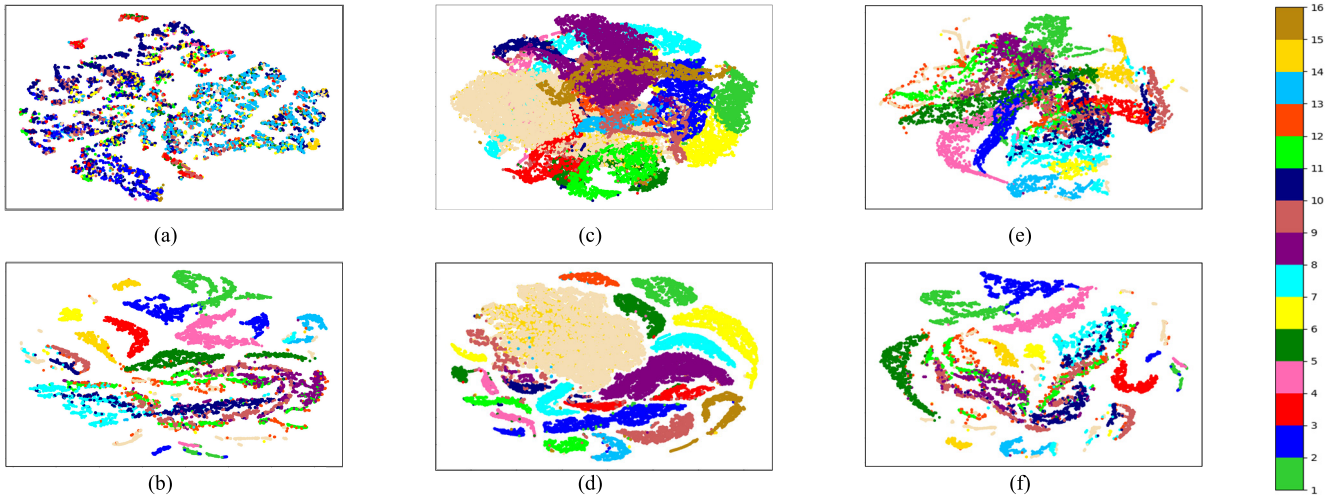


Fig. 6. 2-D visualization of the L2GCC on the IP (first column), Salinas (second column), and UH2013 (third column) at epochs 0 and 200. Different colors in the maps denote different land covers. (a) 3.41%. (b) 73.51%. (c) 25.62%. (d) 83.15%. (e) 10.34%. (f) 64.12%.

clustering methods by learning the high-level semantic features to measure interrelation between raw data. For example, OAs achieved by AE+k-means and CC are 51.80% and 53.48%, respectively, which improves the best performance in shallow methods by 12.04% and 13.72%. Furthermore, compared with the k-means, the deep version of AE+k-means obtains a considerable improvement in terms of clustering accuracy. Third, L2GCC still achieves the best quantitative results, which further validates the superiority of our method design. Generally, the proposed L2GCC method not only enables filter image noise but also has good adaptability to different datasets.

4) *Visual Clustering Results:* The visual clustering map of investigated methods on IP, Salinas, and UH2013 is shown in Figs. 3–5. From the results, we can easily observe that our LGCC achieves the least misclassification clustering maps, which are noticeably closer to the ground truth. In contrast, the comparative methods yield poorer clustering maps with obvious salt and pepper noises. This is because the low-pass graph filter adopted in the L2GCC can preserve the low-frequency signal, which helps for clustering. The layerwise local features learned by LGAT provide rich features for clustering. Furthermore, high-level features learned by L2GCC contribute to achieving a more accurate clustering map.

E. t-SNE Data Distributions Visualization

In this experiment, the t-distributed stochastic neighbor embedding (t-SNE) [43] is adopted to visualize the change of data distributions of IP, Salinas, and UH2013 with L2GCC training. The 2-D visualization of the L2GCC on three datasets at epochs 0 and 200 is shown in Fig. 6. From the results, we can conclude that our L2GCC can well distinguish different types of nodes with the increasing training iteration. Specifically, the small interclass dissimilarity and the large intraclass variations existed in the original three datasets (first row). As network training, L2GCC can learn the most discriminative feature representations, which are both intraclass compact and interclass dispense, and contribute to higher clustering accuracy.

F. Parameter Study

In this section, we investigate the influence of the number of superpixels S (graph node), the number of low-pass graph convolutional layers L, learning rate η, and the number of epochs T on the performance of L2GCC. We divide four parameters into two groups, and a grid search strategy is utilized to find the optimal setting. The experimental setups are consistent with Section V-C, and the clustering accuracy OA is adopted to report the performance of our L2GCC method with different parameter settings. We show the parament-sensitive results of two group parameters on IP, Salians, and UH2013 in Figs. 7 and 8.

1) S and L Effect Analysis: The parament-sensitive results of S and L on L2GCC are shown in Fig. 7. In the experiment, S is varied from 600 to 1400 with an interval of 2500 on the IP dataset, from 3000 to 11000 with an interval of 2000 on the Salinas dataset, and from 7500 to 17500 on the UH2013 dataset, and the L is set from 1 to 5. Furthermore, we fix T and η, as shown in Table II. In the results, the OAs on three datasets are being improved as increase of S. However, the clustering accuracy does not increase after reaching S a certain threshold. The bigger S means smaller superpixels, and thus, we can preserve more local spatial features of HSI for clustering. However, bigger S leads to more graph node, and too many graph nodes will bring challenges to network feature extraction and increase the computational complexity. Therefore, S is 1000, 7000, and 14 000 for IP, Salinas, and UH2013, respectively. In terms of L, a bigger L can learn the high-level features of graphs effectively. However, more weight parameters and more time are required to train the network. In our L2GCC, we can observe that the optimal L is 3 on three datasets.

2) T and η Effect Analysis: In the experiment, we vary T from 50 to 250 with an interval of 50, and η is set as 0.0001,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

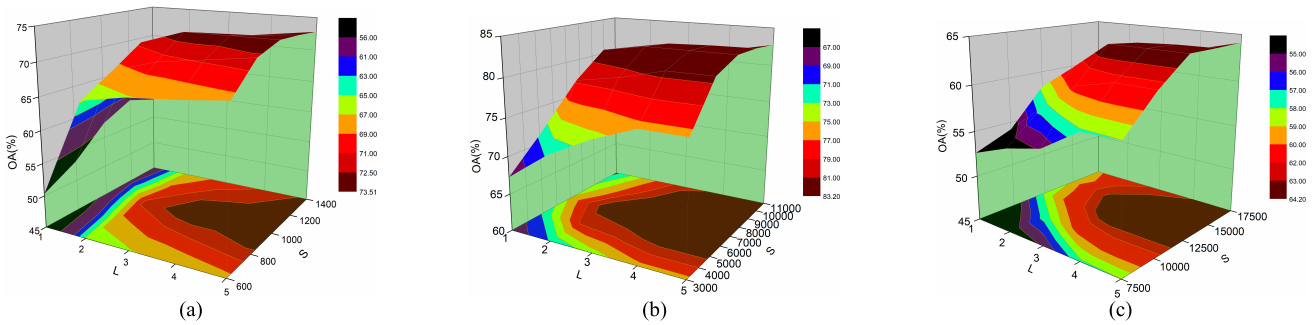


Fig. 7. 3-D visualization of the LGCC with different S and L values on (a) IP, (b) Salinas, and (c) UH2013 datasets.

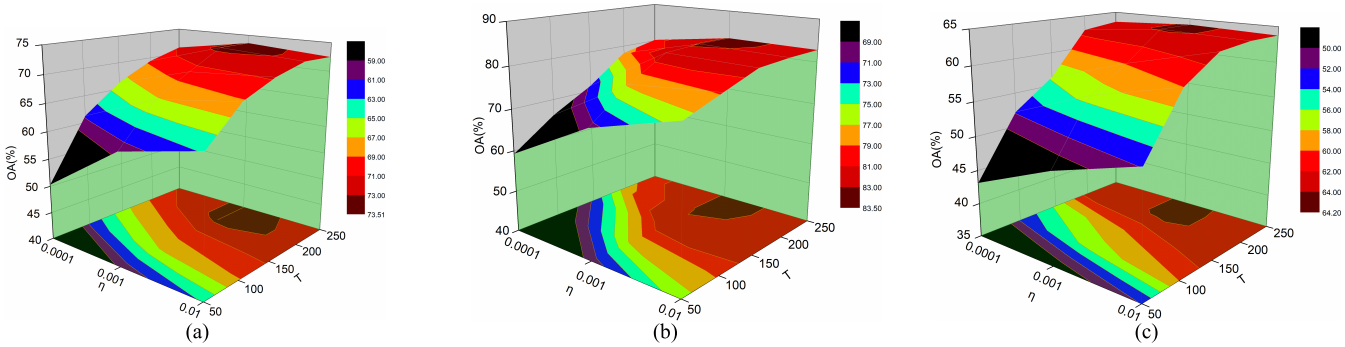


Fig. 8. 3-D visualization of LGCC with different T and η values on (a) IP, (b) Salinas, and (c) UH2013 datasets.

TABLE VI

ABLATION STUDY RESULTS ON IP, SALINAS, AND UH2013. THE OA INDEXES (%) ARE REPORTED. THE PRT MODULE (HSI PREPROCESSING) IS SIMPLIFIED AS "PRT"

Name		L2GCC-v_1	L2GCC-v_2	L2GCC-v_3	L2GCC-v_4	L2GCC-v_5	L2GCC-v_6	L2GCC-v_7	L2GCC
Module	PRT	----	√	----	----	----	√	√	√
	Low-pass GCN	----	----	√	----	√	----	√	√
	LGAT	----	----	----	√	√	√	----	√
Dataset	IP	59.76	65.23	64.26	63.28	66.82	69.35	72.39	**73.51**
	Salinas	66.38	72.34	71.72	70.61	73.39	78.23	82.26	**83.15**
	UH2013	54.19	59.73	59.81	53.33	55.17	58.76	62.75	**64.12**

0.001, and 0.01. Furthermore, S, L, m, and δ are fixed as in Table II. The parament-sensitive results of T and η on L2GCC are shown in Fig. 8. We can train the paraments in the network faster with a large η. However, we usually cannot obtain an optimal solution, and the final results are unstable. Therefore, η is set as 0.001 in our L2GCC method. It should also be noted that T is closely related to η, and a bigger T means needing more time to train the network. From the results, we can observe that the optimal results occur when T is 200.

G. Ablation Study

In this experiment, we assess the role of PRT module, low-pass GCN, and LGAT in L2GCC for boosting the HSI clustering on the IP, Salinas, and UH2013 datasets. The experimental setup is kept identical to Section V-C. The OA is adopted to report the clustering accuracy, and the results are shown in Table VI. For simplicity, we utilize "L2GCC-v_1"–"L2GCC-v_7" to represent the reduced methods. From the results, we know that L2GCC achieves the best performances. Concretely, the clustering accuracies of LGCC-v_2 are better than the values of L2GCC-v_1, and L2GCC-v_7 performs better than L2GCC-v_3, which indicates that PRT can improve the clustering accuracy. Compared with LGCC-v_1, L2GCC-v_3 achieves a superior result, and LGCC-v_5 outperforms LGGCC-v_4, which verifies that low-pass GCN plays an important role in the proposed methods. In the same way, we can observe that the OA of L2GCC-v_3 is higher than that of L2GCC-v_1, and L2GCC-v_6 achieves a superior result to that of L2GCC-v_2, which shows the superiority of LGAT. Therefore, PRT, low-pass GCN, and LGAT contribute to improving the clustering accuracy in our L2GCC.

VI. CONCLUSION

In this article, a novel self-supervised L2GCC framework is developed to boost the clustering performance for large-scale HSI datasets. Specifically, a region transformation module is designed to transform the local spatial–spectral features of the HSI and reduce the computational complexity.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

The locality preserving low-pass graph convolutional embedding autoencoder is developed to efficiently learn the high-level hidden representations at the superpixel level, in which the low-pass GCN and layerwise GAT are designed to extract the smoother features and locality features, respectively. Furthermore, the KL divergence is adopted to evaluate the graph reconstruction loss. We perform the self-training clustering with soft labels generated from the graph embedding. The L2GCC algorithm is jointly characterized by locality preserving low-pass graph convolutional embedding autoencoder and self-training clustering, and both components are mutually beneficial. Compared with state-of-the-art clustering methods, L2GCC shows a remarkable model design and superiority, the clustering accuracy OAs are 73.51%, 83.15%, and 64.12% on IP, Salinas, and UH2013, respectively.

Compared with the supervised and semisupervised methods, the unsupervised methods are needed to be improved in terms of clustering accuracy, especially for large-scale HSIs. However, the success of L2GCC shows the broad prospect of unsupervised clustering methods applied to HSI classification. In the future, we will improve the HSI preprocessing method and focus on exploring more forms of GCN-based self-supervised autoencoder (e.g., GCN-GAN), and so on.

REFERENCES

[1] H. Liu, W. Li, X.-G. Xia, M. Zhang, C.-Z. Gao, and R. Tao, "Central attention network for hyperspectral imagery classification," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Mar. 10, 2022, doi: 10.1109/TNNLS.2022.3155114.

[2] Y. Gao *et al.*, "Hyperspectral and multispectral classification for coastal wetland using depthwise feature interaction network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022.

[3] Y. Ding, X. Zhao, Z. Zhang, W. Cai, and N. Yang, "Graph sample and aggregate-attention network for hyperspectral image classification," *IEEE Geosci. Remote. Sens. Lett.*, vol. 19, pp. 1–5, 2021, doi: 10.1109/LGRS.2021.3062944.

[4] Y. Ding, X. Zhao, Z. Zhang, W. Cai, and N. Yang, "Multiscale graph sample and aggregate network with context-aware learning for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 4561–4572, 2021, doi: 10.1109/JSTARS.2021.3074469.

[5] Y. Ding, X. Zhao, Z. Zhang, W. Cai, N. Yang, and Y. Zhan, "Semi-supervised locality preserving dense graph neural network with ARMA filters and context-aware learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–12, 2021, doi: 10.1109/TGRS.2021.3100578.

[6] W. Li, Y. Gao, M. Zhang, R. Tao, and Q. Du, "Asymmetric feature fusion network for hyperspectral and SAR image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Feb. 18, 2022, doi: 10.1109/TNNLS.2022.3149394.

[7] Y. Zhang, W. Li, R. Tao, J. Peng, Q. Du, and Z. Cai, "Cross-scene hyperspectral image classification with discriminative cooperative alignment," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 9646–9660, Nov. 2021.

[8] D. Yao *et al.*, "Deep hybrid: Multi-graph neural network collaboration for hyperspectral image classification," *Defence Technol.*, early access, Feb. 2022, doi: 10.1016/J.DT.2022.02.007.

[9] Y. Cai, Z. Zhang, Z. Cai, X. Liu, X. Jiang, and Q. Yan, "Graph convolutional subspace clustering: A robust subspace clustering framework for hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 5, pp. 4191–4202, May 2021.

[10] R. Vidal, "Subspace clustering," *IEEE Signal Process. Mag.*, vol. 28, no. 2, pp. 52–68, Mar. 2011.

[11] J. Lei, X. Li, B. Peng, L. Fang, N. Ling, and Q. Huang, "Deep spatial–spectral subspace clustering for hyperspectral image," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 7, pp. 2686–2697, Jul. 2021.

[12] X. Peng, J. Feng, J. T. Zhou, Y. Lei, and S. Yan, "Deep subspace clustering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 12, pp. 5509–5521, Dec. 2020.

[13] X. Yang, G. Lin, Y. Liu, F. Nie, and L. Lin, "Fast spectral embedded clustering based on structured graph learning for large-scale hyperspectral image," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[14] R. Wang, F. Nie, and W. Yu, "Fast spectral clustering with anchor graph for large hyperspectral images," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 11, pp. 2003–2007, Nov. 2017.

[15] T. Kanungo, D. M. Mount, N. S. Netanyahu, C. D. Piatko, R. Silverman, and A. Y. Wu, "An efficient K-means clustering algorithm: Analysis and implementation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 881–892, Jul. 2002.

[16] A. Y. Ng, M. I. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2002, pp. 849–856.

[17] T.-N. Yang, C.-J. Lee, and S.-J. Yen, "Fuzzy objective functions for robust pattern recognition," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, Aug. 2009, pp. 2057–2062.

[18] C. A. Shah, M. K. Arora, and P. K. Varshney, "Unsupervised classification of hyperspectral data: An ICA mixture model-based approach," *Int. J. Remote Sens.*, vol. 25, no. 2, pp. 481–487, Jan. 2004.

[19] Y. Zhang, W. Li, M. Zhang, Y. Qu, R. Tao, and H. Qi, "Topological structure and semantic information transfer network for cross-scene hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Sep. 16, 2021, doi: 10.1109/TNNLS.2021.3109872.

[20] J. Lin, C. He, Z. J. Wang, and S. Li, "Structure preserving transfer learning for unsupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 10, pp. 1656–1660, Oct. 2017.

[21] J. M. Murphy and M. Maggioni, "Unsupervised clustering and active learning of hyperspectral images with nonlinear diffusion," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 3, pp. 1829–1845, Mar. 2019.

[22] E. Elhamifar and R. Vidal, "Sparse subspace clustering: Algorithm, theory, and applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 11, pp. 2765–2781, Nov. 2013.

[23] X. Peng, H. Tang, L. Zhang, Z. Yi, and S. Xiao, "A unified framework for representation-based subspace clustering of out-of-sample and large-scale data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 12, pp. 2499–2512, Dec. 2016.

[24] J. Fan, T. Chen, and S. Lu, "Superpixel guided deep-sparse-representation learning for hyperspectral image classification," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 28, no. 11, pp. 3163–3173, Nov. 2018.

[25] S. Matsushima and M. Brbic, "Selective sampling-based scalable sparse subspace clustering," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alche-Buc, E. Fox, and R. Garnett, Eds. San Jose, CA, USA: Curran Associates, 2019.

[26] H. Zhai, H. Zhang, L. Zhang, and P. Li, "Total variation regularized collaborative representation clustering with a locally adaptive dictionary for hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 1, pp. 166–180, Aug. 2018.

[27] S. Chen and D. Zhang, "Robust image segmentation using FCM with spatial constraints based on new kernel-induced distance measure," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 34, no. 4, pp. 1907–1916, Aug. 2004.

[28] Y. Ding *et al.*, "AF2GNN: Graph convolution with adaptive filters and aggregator fusion for hyperspectral image classification," *Inf. Sci.*, vol. 602, pp. 201–219, Jul. 2022.

[29] F. Nie, X. Wang, M. Jordan, and H. Huang, "The constrained Laplacian rank algorithm for graph-based clustering," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, no. 1, 2016, pp. 1–8.

[30] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *Proc. 33rd Int. Conf. Mach. Learn. Res.*, vol. 48, M. F. Balcan and K. Q. Weinberger, Eds. New York, NY, USA, Jun. 2016, pp. 478–487.

[31] Y. Cai, M. Zeng, Z. Cai, X. Liu, and Z. Zhang, "Graph regularized residual subspace clustering network for hyperspectral image clustering," *Inf. Sci.*, vol. 578, pp. 85–101, Nov. 2021.

[32] Y. Cai *et al.*, "Superpixel contracted neighborhood contrastive subspace clustering network for hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2022.

[33] S. Hou, H. Shi, X. Cao, X. Zhang, and L. Jiao, "Hyperspectral imagery classification based on contrastive learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2022.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.

[34] Y. Li, P. Hu, Z. Liu, D. Peng, J. T. Zhou, and X. Peng, "Contrastive clustering," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, May 2021, vol. 35, no. 10, pp. 8547–8555.

[35] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and locally connected networks on graphs," 2013, *arXiv:1312.6203*.

[36] T. N. Kipf and M. Welling, "Variational graph auto-encoders," 2016, *arXiv:1611.07308*.

[37] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017, pp. 1–4.

[38] P. Veličković *et al.*, "Graph attention networks," 2017, *arXiv:1710.10903*.

[39] C. Wang, S. Pan, G. Long, X. Zhu, and J. Jing, "MGAE: Marginalized graph autoencoder for graph clustering," in *Proc. ACM Conf. Inf. Knowl. Manag.*, 2017, pp. 1–10.

[40] J. Park, M. Lee, H. J. Chang, K. Lee, and J. Y. Choi, "Symmetric graph convolutional autoencoder for unsupervised graph representation learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 6519–6528.

[41] X. Zhang, H. Liu, Q. Li, and X.-M. Wu, "Attributed graph clustering via adaptive graph convolution," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 4327–4333.

[42] S. Pan, R. Hu, G. Long, J. Jiang, L. Yao, and C. Zhang, "Adversarially regularized graph autoencoder for graph embedding," in *Proc. IJCAI*, 2018, pp. 2609–2615.

[43] R. Horn and C. Johnson, *Matrix Analysis*. Cambridge, U.K.: Cambridge Univ. Press, 2012.

[44] B. Schölkopf, A. Smola, and K.-R. Müller, "Nonlinear component analysis as a kernel eigenvalue problem," *Neural Comput.*, vol. 10, no. 5, pp. 1299–1319, Jul. 1998.

[45] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.

[46] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[47] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 1–27, 2008.



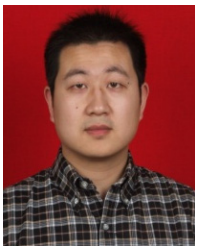
Yao Ding received the M.S. degree from the Key Laboratory of Optical Engineering, Xi'an Research Institute of High Technology, Xi'an, China, in 2013, where he is currently pursuing the Ph.D. degree with the School of Key Laboratory of Optical Engineering.

His research interests include neural networks, computer vision, image processing, and hyperspectral image clustering.



Zhili Zhang received the M.S. and Ph.D. degrees from the Xi'an Research Institute of High Technology, Xi'an, China, in 1991 and 2001, respectively.

He is currently a Professor with the School of National Key Laboratory of Optical Engineering, Xi'an Research Institute of High Technology. He has published more than 300 academic articles. His research areas include optical engineering, image processing, computer vision, and pattern recognition.



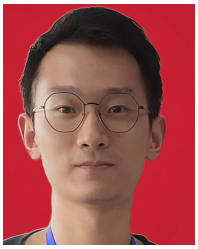
Xiaofeng Zhao received the B.S. degree in applied physics and the Ph.D. degree in optics from Wuhan University, Wuhan, Hubei, China, in 2001 and 2006, respectively.

He is currently a Professor with the School of National Key Laboratory of Optical Engineering. His research areas include machine learning, image processing, computer vision, and target recognition.



Yaoming Cai (Member, IEEE) received the B.Eng. degree in information security from the China University of Geosciences, Wuhan, China, in 2016, where he is currently pursuing the Ph.D. degree with the School of Computer Science.

From August 2021 to August 2022, he was a Visiting Ph.D. Student with Helmholtz-Zentrum Dresden-Rossendorf, Helmholtz Institute Freiberg or Resource Technology, Freiberg, Germany. His research interests include machine learning, evolutionary computation, and remote sensing image processing.



Siye Li received the B.S. degree from the Xi'an Research Institute of High Technology, Xi'an, China, where he is currently pursuing the Ph.D. degree in aeronautical and astronautical science and technology.

His research interests include machine learning, hyperspectral image analysis, and data-driven simulations.



Biao Deng received the Ph.D. degree in electromechanical integration from Xi'an Jiaotong University, Xi'an, China, in 2011.

He is currently a Professor with the Vehicle Evaluation Group, Xi'an Research Institute of High Technology, Xi'an. His main research interests include electromechanical system virtual design and simulation, complex system virtual maintenance analysis, and evaluation of vehicle viability.



Weiwei Cai (Member, IEEE) is currently with the School of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi, China, and the Graduate School, Northern Arizona University, Flagstaff, AZ, USA. Prior to that, he worked with IT industry for more than ten years in the roles of a system architect and the program manager.

Mr. Cai is an Associate Editor of *Journal of Cloud Computing* and an Editorial Board Member of *Wireless Communications and Mobile Computing*. He has served as a Guest Editor for the *CMES-Computer Modeling in Engineering & Sciences* and *Information*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 04:21:54 UTC from IEEE Xplore. Restrictions apply.